

Research Road Map of Relaxed System Lab

Binhang Yuan

03.05.26

Amazing Progress of AIGC.

 DeepSeek-R1 is now live and open source, rivaling OpenAI's Model o1. Available on web, app, and API. [Click for details.](#)



deepseek

Into the unknown

Start Now

Free access to DeepSeek-V3.
Experience the intelligent model.

Get DeepSeek App

Chat on the go with DeepSeek-V3
Your free all-in-one AI tool

OpenAI

March 5, 2026 Product Release

Introducing GPT-5.4

Designed for professional work

 Listen to article

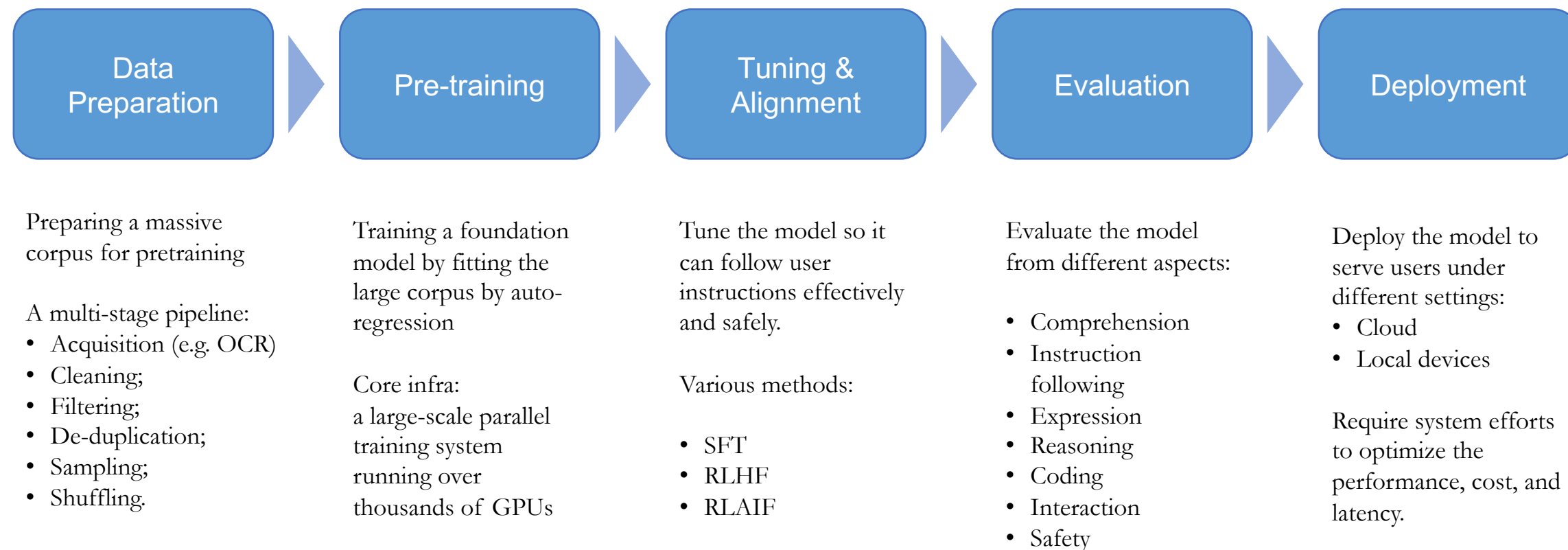
16:00

 Share

Today, we're releasing **GPT-5.4** in ChatGPT (as GPT-5.4 Thinking), the API, and Codex. It's our most capable and efficient frontier model for professional work. We're also releasing **GPT-5.4 Pro** in ChatGPT and the API, for people who want maximum performance on complex tasks.



The Path To a Foundation Model

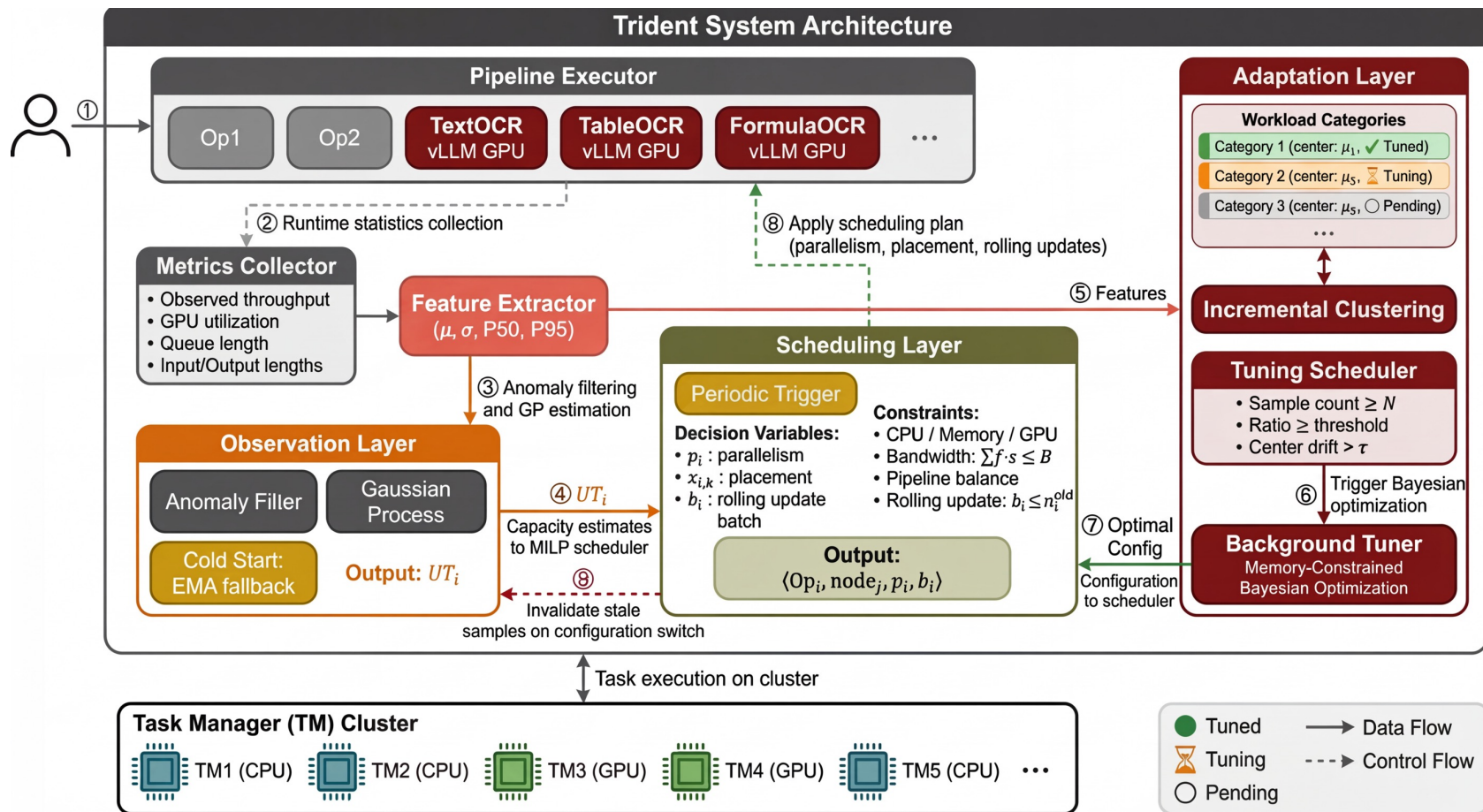




*Accommodate LLM training through
heterogeneous hardware.*

Trident

Adaptive Scheduling for Heterogeneous Multimodal Data Pipelines



Trident: Adaptive Scheduling for Heterogeneous Multimodal Data Pipelines

Ding Pan¹, Zhuangzhuang Zhou², Long Qian², Binhang Yuan¹

¹HKUST, ²Huawei

Abstract

The rapid adoption of large language models and multimodal foundation models has created unprecedented demand for high-quality training data, making multimodal data preparation pipelines a critical AI infrastructure. These pipelines interleave CPU-heavy preprocessing with AI accelerator-backed (e.g., GPU/NPU/TPU) inference operators and process massive intermediate artifacts. Achieving high throughput is difficult for such a computation paradigm, where the multimodal workloads are highly non-stationary: workload regime shifts and input-dependent inference behavior cause rapid performance fluctuations, while transient memory spikes can trigger out-of-memory (OOM) failures. Existing schedulers often rely on threshold-based autoscaling or assume synchronous, homogeneous operators, leading to severe inefficiency.

To overcome these essential challenges, we introduce TRIDENT, an adaptive scheduling framework for heterogeneous multimodal data pipelines on fixed-resource clusters, integrating three tightly coupled layers in a closed loop: (i) an *observation layer* that provides noise-resilient capacity estimates for asynchronous operators using Gaussian Process regression with two-stage anomaly filtering; (ii) an *adaptation layer* that tracks workload regime shifts via online clustering and uses memory-constrained Bayesian optimization to tune operator configurations sample-efficiently, reducing OOM risk; (iii) a *scheduling layer* formulates a mixed-integer linear program (MILP) that jointly optimizes operator parallelism, placement, and configuration transitions under heterogeneous compute and bandwidth constraints, trading off throughput gains against cold-start overhead via rolling updates. The three layers form a closed control loop: capacity estimates and configuration recommendations flow into the MILP, whose decisions in turn trigger sample invalidation and model updates, ensuring system-wide consistency. We evaluate TRIDENT on production-representative document and video curation pipelines. TRIDENT improves end-to-end throughput by up to 2.01 $\times$ (PDF pipeline) and 1.88 $\times$ (video pipeline) over a static baseline, substantially outperforming existing schedulers, with low overhead suitable for online re-optimization.

1 Introduction

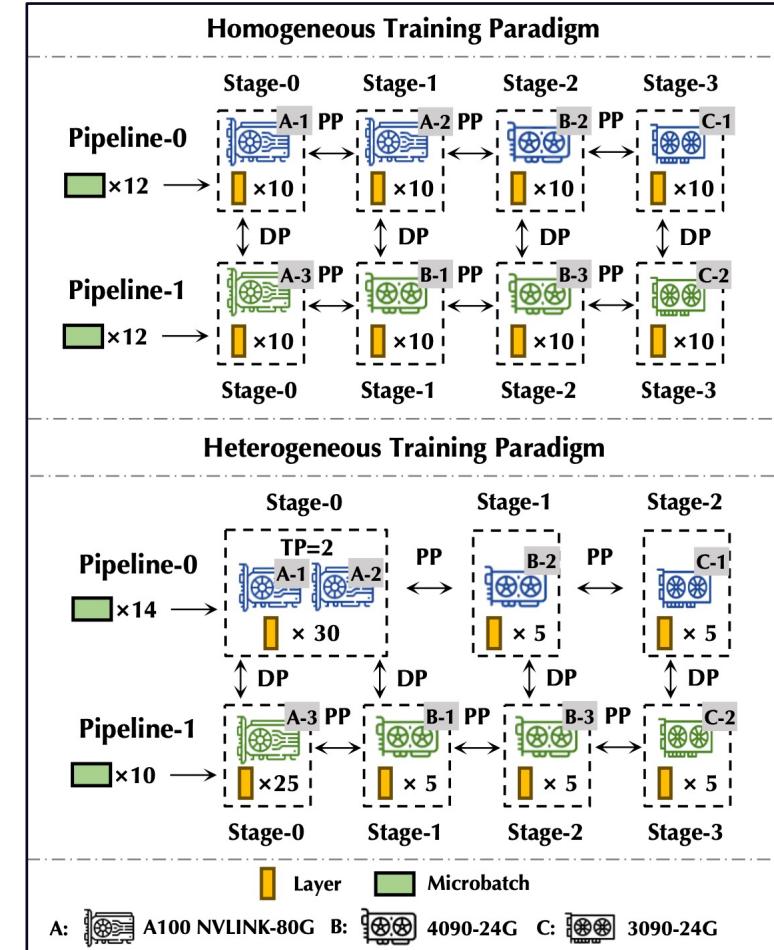
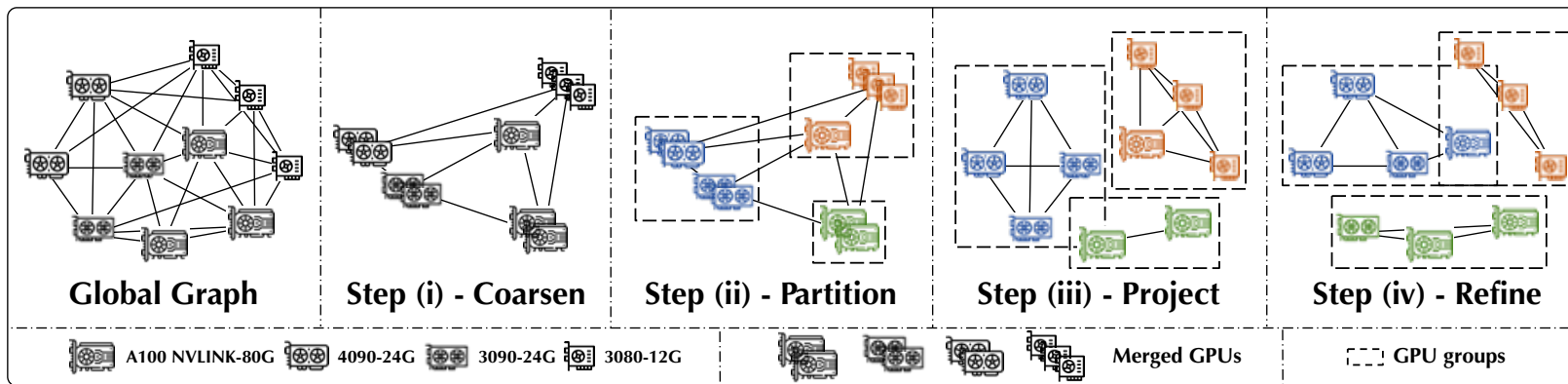
The rapid advancement of large language models (LLMs) and multimodal foundation models (MPMs) has created an unprecedented demand for high-quality training data [1–3]. Multimodal data preparation pipelines that process petabytes of documents [2, 4], images [5], and videos [6] have become a critical infrastructure for modern generative AI development. These pipelines contain various operators with heterogeneous resource utilization, such as video decoding, filtering operators using CPUs, alongside LLM-based assessment operators running on AI accelerators such as GPUs, TPUs, or NPUs, where a large volume of intermediate multimodal data flows through the pipelines.

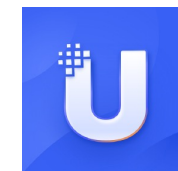
[Arxiv. 2603.02075]

arXiv:2603.02075v1 [cs.LG] 2 Mar 2026



- A heterogeneous LLM training system that supports:
 - Data parallelism;
 - Pipeline parallelism;
 - Tensor model parallelism;
- A scheduling algorithm:
 - Two-phase graph partition algorithm





- A heterogeneous LLM training system that supports:
 - Data parallelism;
 - Pipeline parallelism;
 - Tensor model parallelism;
- A scheduling algorithm:
 - Two-phase graph partition algorithm



<https://github.com/Relaxed-System-Lab/HexiScale>

HEXISCALE: FACILITATING LARGE LANGUAGE MODEL TRAINING OVER HETEROGENEOUS HARDWARE

Ran Yan^{*1} Youhe Jiang^{*1} Xiaonan Nie² Fangcheng Fu³ Bin Cui² Binhang Yuan¹

ABSTRACT

Training large language models (LLMs) is a computationally intensive task, which is typically conducted in data centers with homogeneous high-performance GPUs. In this paper, we explore an alternative approach by deploying training computations across heterogeneous GPUs to enable better flexibility and efficiency for heterogeneous resource utilization. Toward this end, we propose a novel system, HEXISCALE, that can flexibly support asymmetric partition of training computations in the scope of data-, pipeline-, and tensor model parallelism. We further formalize the allocation of asymmetric partitioned training computations over a set of heterogeneous GPUs as a constrained optimization problem and propose an efficient hierarchical graph partitioning algorithm. Our approach effectively allocates training computations across heterogeneous GPUs, fully leveraging the available computational power. We compare the performance of HEXISCALE with state-of-the-art homogeneous and heterogeneous training systems. When training LLMs at different scales (from 7B to 30B), empirical results demonstrate that: (i) compared to state-of-the-art homogeneous baselines running over homogeneous GPUs, HEXISCALE achieves *similar* performance when running over heterogeneous GPUs with the *same* theoretical FLOPS; (ii) compared to state-of-the-art heterogeneous baselines running on the same heterogeneous clusters, HEXISCALE delivers $1.5\times$ to $2.4\times$ higher throughput.

1 INTRODUCTION

Over the past few years, large language models (LLM) have demonstrated impressive performance and sparked a new wave of exciting AI applications (Bommasani et al., 2021). However, training these LLMs, such as GPT (OpenAI, 2024), Claude (Anthropic, 2024), Gemini (Reid et al., 2024), Llama (Touvron et al., 2023; Dubey et al., 2024), Mixtral (Jiang et al., 2024a), Yi (Young et al., 2024), Falcon (Institute, 2023) etc., can be extremely computation-intensive, often involving thousands of GPUs running for months. The high cost of deploying such training tasks in a cluster with homogeneous GPUs has become an obvious obstacle limiting the evolution of LLMs. We explore an alternative approach by *distributing training computations across heterogeneous GPUs*, enabling greater flexibility in resource utilization and democratizing LLM training.

Distributing parallel training computations across heterogeneous GPUs is a natural option to democratize LLM training.

^{*}Equal contribution ¹Department of Computer Science and Engineering, The Hong Kong University of Science and Technology, Hong Kong, China ²School of Computer Science, Peking University, Beijing, China ³School of Artificial Intelligence, Shanghai Jiao Tong University, Shanghai, China. Correspondence to: Binhang Yuan <biyuan@ust.hk>.

In the current exciting era of generative AI, chip vendors typically release new generations of AI chips every 24 months. For instance, Nvidia introduced the Turing architecture in 2018 (Nvidia, 2018), Ampere in 2020 (Nvidia, 2020), Hopper in 2022 (Nvidia, 2022), and Blackwell is scheduled for Q4, 2024 (Nvidia, 2024). On the other hand, one particular version of an AI chip often remains in use by cloud service platforms, technology companies, or research institutions for a much longer period. For example, K80 GPUs with Tesla architecture, released in 2006 (Nvidia, 2006), are still available on AWS as p2 instances (Amazon, 2024). This observation highlights the important opportunity to explore effective ways to maximize efficiency of such widely available yet heterogeneous hardware to facilitate more cost-effective and accessible LLM training.

On the other hand, deploying large-scale training computation for LLM over a set of heterogeneous GPUs with different technique specs would be a challenging task regarding training system design and implementation. To effectively distribute the training computation over thousands of GPUs, the state-of-the-art training systems, like Megatron (Narayanan et al., 2021b) and DeepSpeed (Rajbhandari et al., 2020) usually supports: (i) tensor model parallelism (Narayanan et al., 2021b; Nagrecha, 2021); (ii) pipeline parallelism (Huang et al., 2019; Narayanan et al., 2019; Yang et al., 2021; Narayanan et al., 2021a); and (iii)

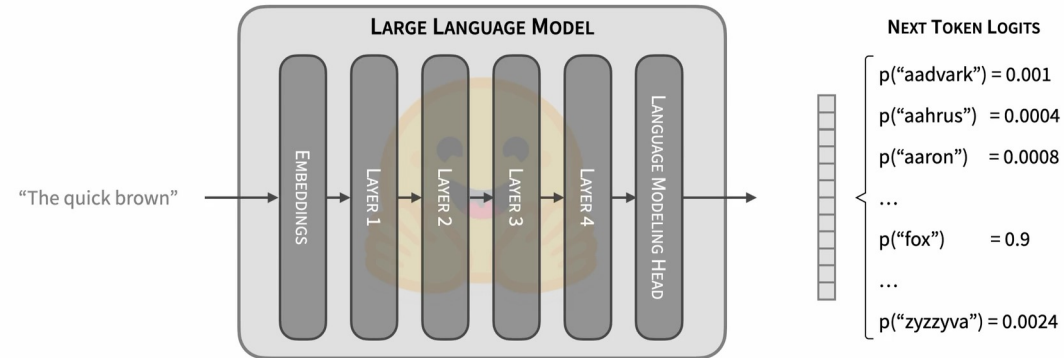
LLM service is NOT all about training.

“90% of the machine learning demand in the cloud is for inference.”

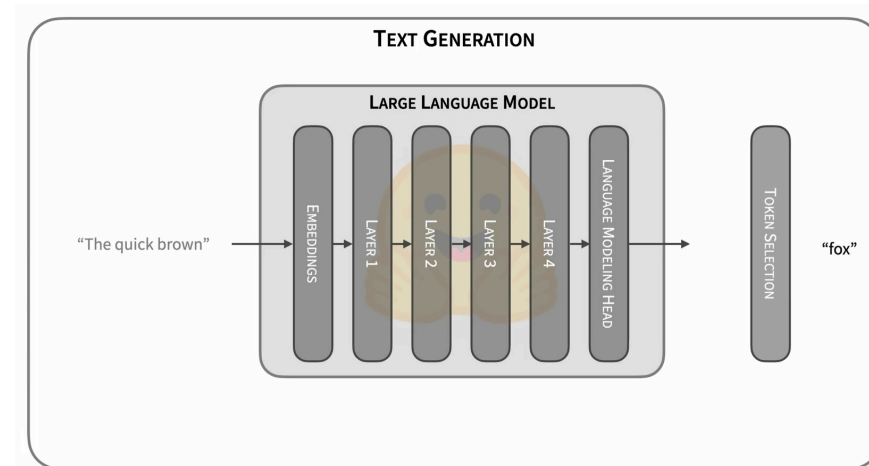
-- AWS Report

Autoregressive Generation

Prefill phase: the model takes a prompt sequence as input and engages in the generation of a key-value cache (KV cache) for each Transformer layer.

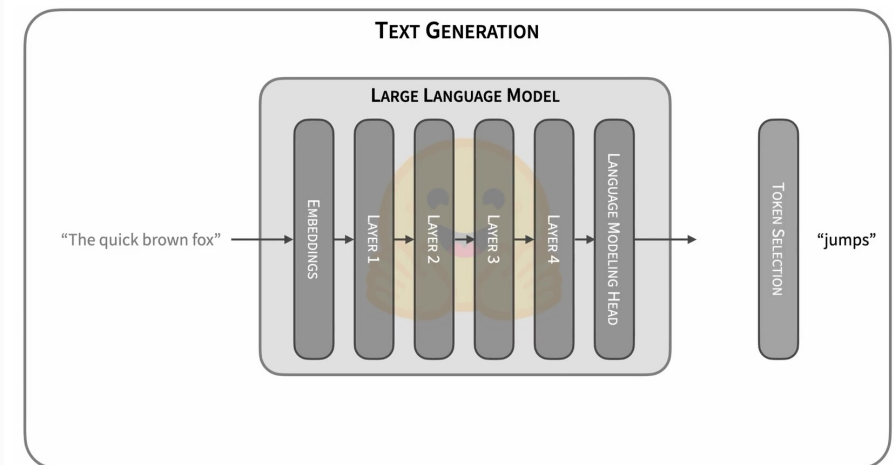


Decode phase: for each decode step, the model updates the KV cache and reuses the KV to compute the output.



The quick brown => fox

Decode step 1



The quick brown fox => jumps

Decode step 2

HexGen: Accommodating LLM Inference over Heterogeneity

- HexGen: schedule the generative inference under the colocating paradigm;
- HexGen-2: schedule the generative inference under the disaggregated paradigm;

HEXGEN: Generative Inference of Large Language Model over Heterogeneous Environment

Youhe Jiang^{*1} Ran Yan^{*1} Xiaozhe Yao^{*2} Yang Zhou³ Beidi Chen³ Binhang Yuan¹

Abstract

Serving generative inference of the large language model is a crucial component of contemporary AI applications. This paper focuses on deploying such services in a heterogeneous and cross-datacenter setting to mitigate the substantial inference costs typically associated with a single centralized datacenter. Towards this end, we propose HEXGEN, a *flexible distributed inference engine* that uniquely supports the asymmetric partition of generative inference computations over both tensor model parallelism and pipeline parallelism and allows for effective deployment across diverse GPUs interconnected by a fully heterogeneous network. We further propose a *sophisticated scheduling algorithm* grounded in constrained optimization that can adaptively assign asymmetric inference computation across the GPUs to fulfill inference requests while maintaining acceptable latency levels. We conduct an extensive evaluation to verify the efficiency of HEXGEN by serving the state-of-the-art LLaMA-2 (70B) model. The results suggest that HEXGEN can choose to achieve up to 2.3× lower latency deadlines or tolerate up to 4× more request rates compared with the homogeneous baseline given the same budget. Our implementation is available at <https://github.com/Relaxed-System-Lab/HexGen>.

1. Introduction

Large language models are distinguished by the vast scale of parameters being trained over a substantial pre-train cor-

^{*}Equal contribution ¹Department of Computer Science and Engineering, The Hong Kong University of Science and Technology, Hong Kong, China ²Department of Computer Science, ETH Zurich, Zurich, Switzerland ³Department of Electrical and Computer Engineering, Carnegie Mellon University, Pittsburgh, Pennsylvania. Correspondence to: Binhang Yuan <byuan@ust.hk>.

Proceedings of the 41st International Conference on Machine Learning, Vienna, Austria. PMLR 235, 2024. Copyright 2024 by the author(s).

HEXGEN-2: DISAGGREGATED GENERATIVE INFERENCE OF LLMs IN HETEROGENEOUS ENVIRONMENT

Youhe Jiang^{*} Ran Yan^{*} Binhang Yuan^{*}

Abstract

Disaggregating the prefill and decoding phases represents an effective new paradigm for generative inference of large language models (LLM), which eliminates prefill-decoding interference and optimizes resource allocation. However, it is still an open problem about how to deploy the disaggregated inference paradigm across a group of heterogeneous GPUs, which can be an economical alternative to deployment over homogeneous high-performance GPUs. Towards this end, we introduce HEXGEN-2, a distributed system for efficient and economical LLM serving on heterogeneous GPUs following the disaggregated paradigm. Built on top of HEXGEN, the core component of HEXGEN-2 is a *scheduling algorithm* that formalizes the allocation of disaggregated LLM inference computations and communications over heterogeneous GPUs and network connections as a constraint optimization problem. We leverage the *graph partitioning* and *max-flow* algorithms to co-optimize resource allocation, parallel strategies for distinct inference phases, and the efficiency of inter-phase key-value (KV) cache communications. We conduct extensive experiments to evaluate HEXGEN-2, i.e., on OPT (30B) and LLaMA-2 (70B) models in various real-world settings, the results reveal that HEXGEN-2 delivers up to a 2.0× and on average a 1.3× improvement in serving throughput, reduces the average inference latency by 1.5× compared with state-of-the-art systems given the same price budget, and achieves comparable inference performance with a 30% lower price budget.

1 INTRODUCTION

Large Language Models (LLMs), such as OPT (Zhang et al., 2022), LLaMA (Touvron et al., 2023), GPT (OpenAI, 2024), GEMINI (Reid et al., 2024), CLAUDE (Anthropic, 2024) and MIXTRAI (Jiang et al., 2024a) have shown exceptional performance across various advanced applications. However, deploying the generative inference service for such LLMs can be costly, typically requiring a substantial number of homogeneous, high-performance GPUs to meet the service demands, such as first token latency and generation throughput. In this paper, we explore an alternative solution that *deploys the most advanced disaggregated generative inference paradigm over a set of heterogeneous GPUs to provide an efficient and economical LLM service*.

Disaggregated inference is currently the most *efficient* framework for serving the generative inference requests of LLMs (Zhong et al., 2024; Patel et al., 2024). By splitting the prefill phase (compute-bounded) and decoding phase (HBM IO-bounded) across different GPUs, the disaggregation significantly reduces interference between different requests and enables more flexible parallel configurations for the two phases. When compared with colocating the prefill and decoding computations, the disaggregated approach optimizes resource usage and enhances the scalability and efficiency of the LLM inference service. Recent efforts (Jiang et al., 2024b; Griggs et al., 2024; Zhao et al., 2024; Miao et al., 2024) have shown that serving LLMs with heterogeneous GPUs can be a *economical* alternative to deploying over homogeneous high-performance GPUs. Heterogeneous deployments offer significant opportunities to reduce inference service costs by leveraging the wide availability of diverse GPU types across commercial and private computing platforms. Note that Nvidia typically releases new GPU generations every 24 months, e.g., Turing in 2018, Ampere in

^{*}Equal contribution

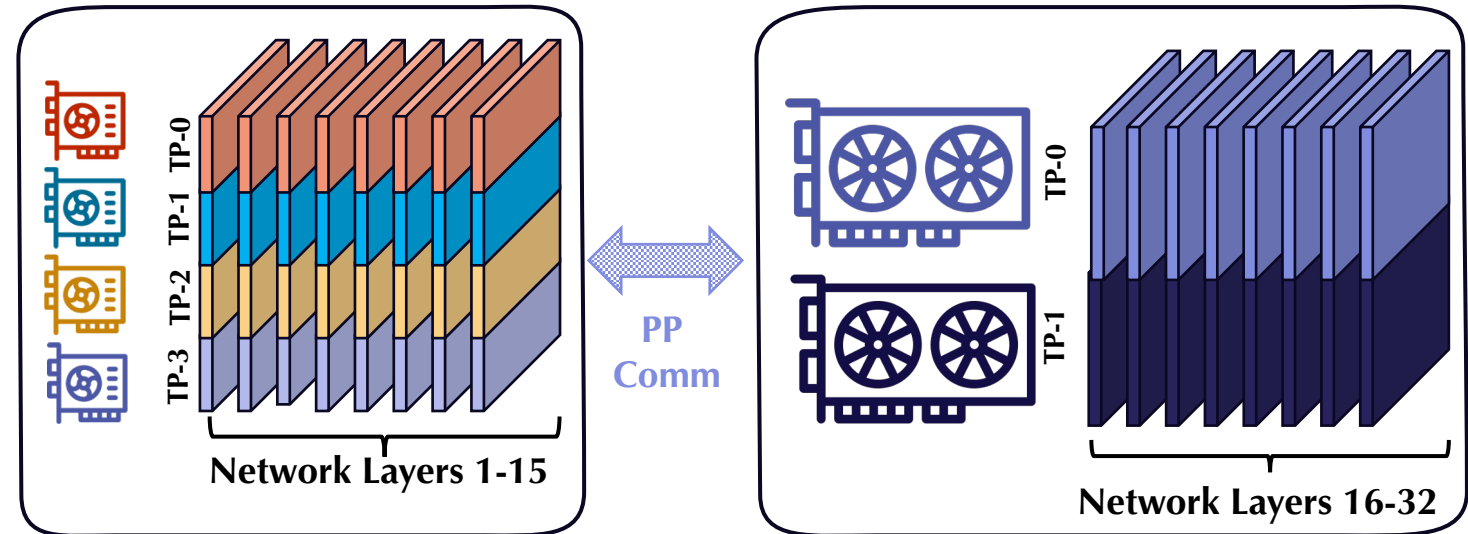
HexGen

Generative Inference of Large Language Model over Heterogeneous Environment

- An implementation that accommodates tensor model parallelism and pipeline parallelism.
- A scheduling algorithm that optimizes pipeline partitions and parallel strategies over heterogeneous GPUs.
 - Optimizing the layout of a pipeline through dynamic programming;
 - Solve the global scheduling through a genetic algorithm.



<https://github.com/Relaxed-System-Lab/HexGen>



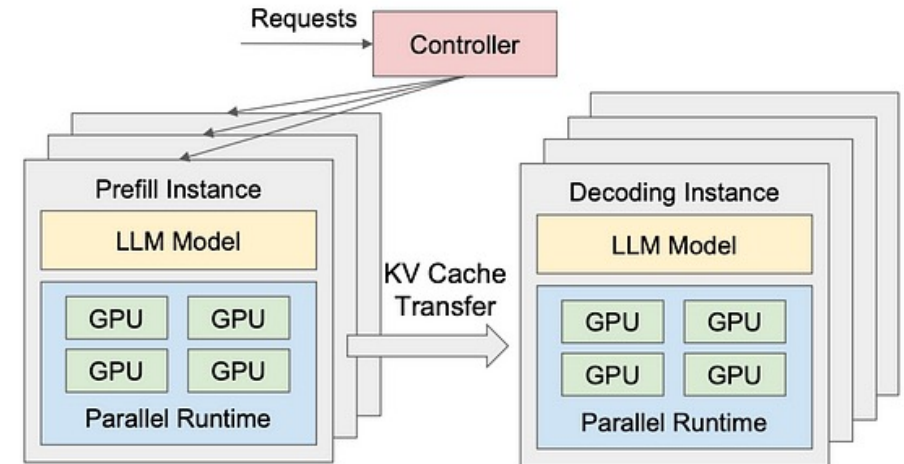
Disaggregated Inference

- **Key ideas:**

- Prefill computation on some GPUs;
- Decoding computation on some other GPUs;
- Prefill and decoding instances can have different parallel configurations;
- Dynamic configuration of the prefill / decoding ratio;
- Overhead: KV-cache communication.

- **Frameworks:**

- DistServe, Splitwise.

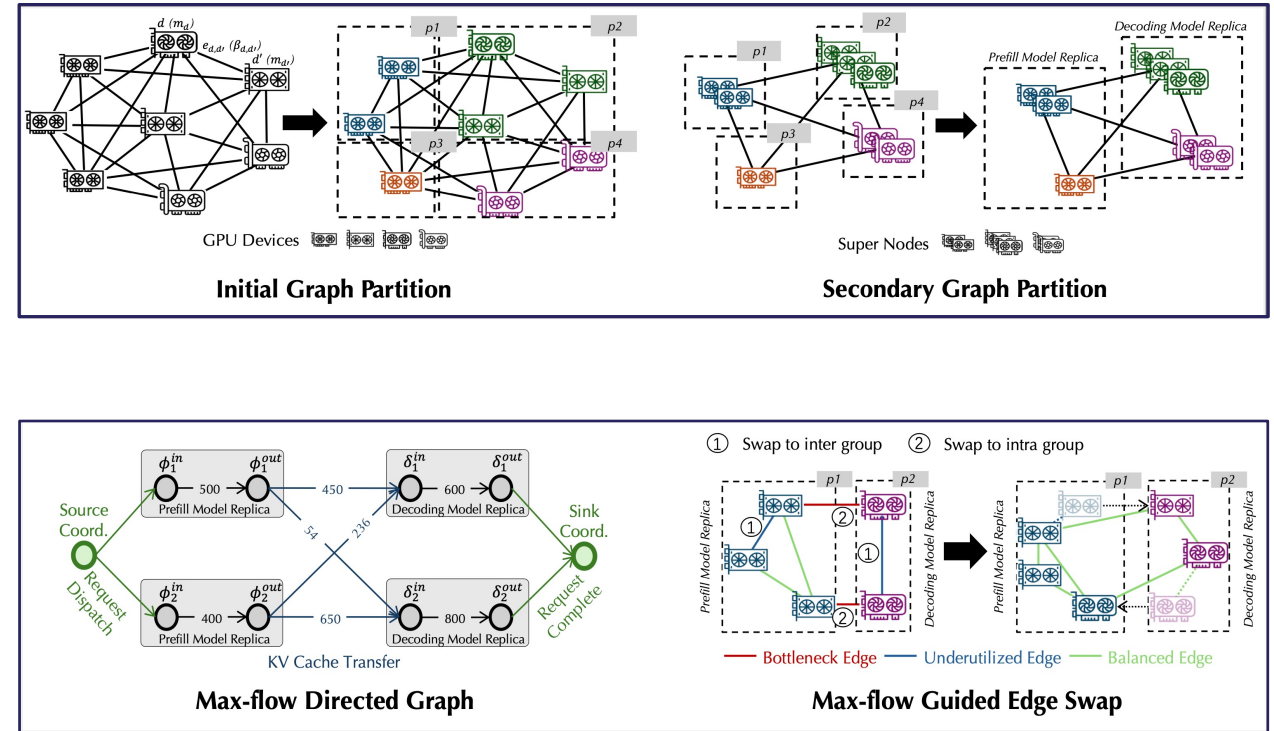


<https://github.com/LLMServe/DistServe>

HexGen-2

Disaggregated Generative Inference of LLMs in Heterogeneous Environments

- Scheduling for the disaggregated framework:
 - **Graph-partition**: partition the set of heterogeneous GPUs into multiple model serving groups, where each group could serve a prefill or a decoding phase;
 - **Max-flow**: find the current optimal parallel strategies for prefill and decoding model replicas and generate the optimal KV cache communication strategy among them;
 - **Iterative refinement**: we iteratively repeat the two-phase algorithm to find the optimal model placement strategy.



Attention – FFN Disaggregated Inference

ATTENTION

Memory-bandwidth bound

- Decode arithmetic intensity $\approx 1\text{--}10$ FLOP/byte
- KV-cache I/O dominates per-token latency
- Cost scales with $\text{seq_len} \times \text{concurrency}$
- Wants: large HBM, high BW, MQA / GQA / paged-KV

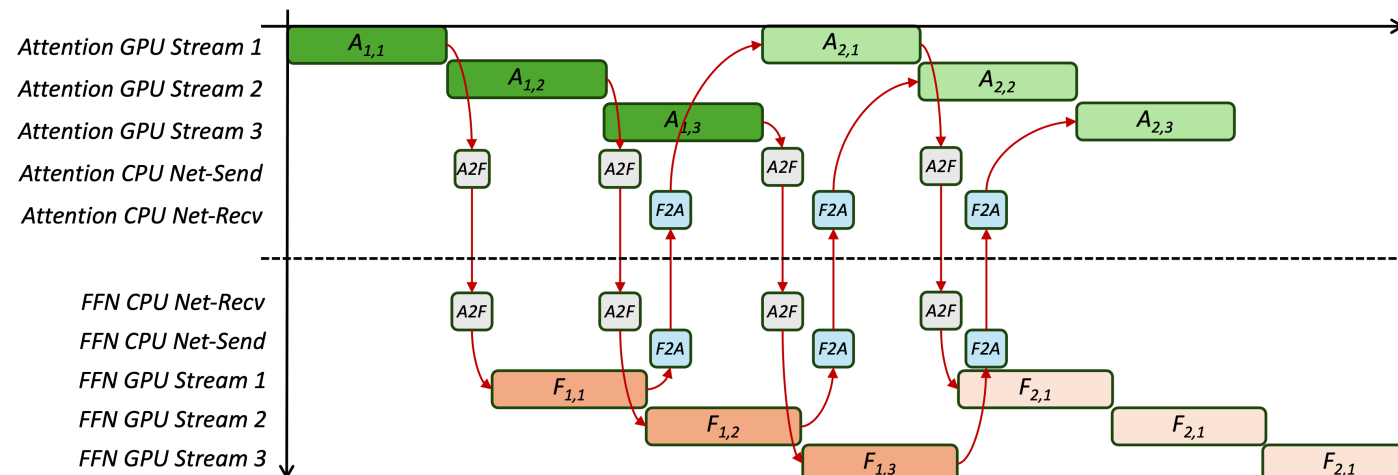
FIT memory-tier GPUs · MI300X · H200 · older nodes with spare HBM

FFN / EXPERTS

Compute / FLOPS bound

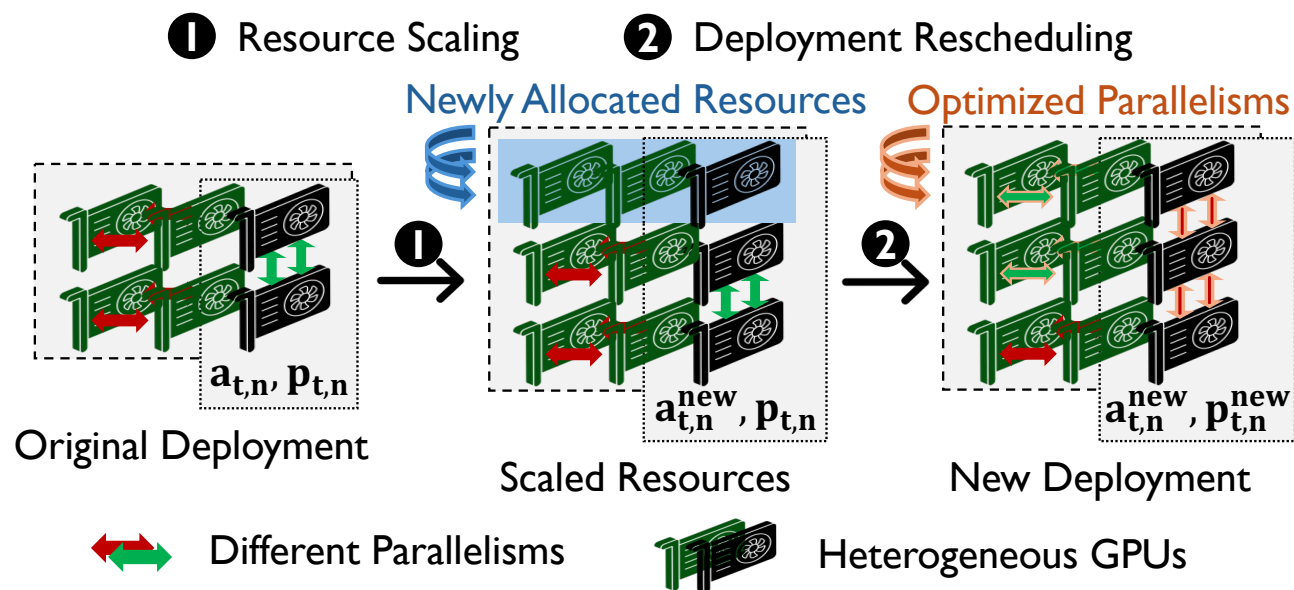
- Dense GEMMs
- High arithmetic intensity $\rightarrow$ loves large batch
- Stateless: no per-request memory footprint
- Wants: high TP, EP for MoE, fast NVLink fabric

FIT compute-tier GPUs · H100 / H200 / B200 · TPU v5p · MoE expert farms



HexGen-3

A Fully Disaggregated LLM Serving Framework with Fine-Grained Heterogeneous Resource Autoscaling



HEXGEN-3: A Fully Disaggregated LLM Serving Framework with Fine-Grained Heterogeneous Resource Autoscaling

Youhe Jiang^{*1} Wenshuang Li^{*1} You Peng¹ Jintao Zhang² Ran Yan¹ Jianfei Chen² Xu Han²
Fangcheng Fu² Binhang Yuan¹

Abstract

The operational cost of serving large language models remains prohibitively high, largely due to extreme *workload heterogeneity* in production traffic. We observe that combining disaggregated inference with resource autoscaling enables *fine-grained* resource adjustment, allowing inference phases and operations to scale independently based on their specific bottlenecks. Building on this insight, we propose HEXGEN-3, a cost-effective LLM serving framework that leverages a fully disaggregated inference architecture and heterogeneous resource autoscaling. HEXGEN-3 introduces two key components: (i) A *hierarchical scheduling framework* that jointly optimizes resource allocation and parallelism configuration for *any* given resource provisioning, and (ii) an *autoscaling framework* that dynamically adjusts resources and triggers deployment rescheduling in response to workload fluctuations. Experiments comparing HEXGEN-3 against state-of-the-art LLM serving systems demonstrate up to 60% (on average 46.5%) improvement in per-cost throughput under static resource provisioning, and up to 78.3% (on average 55.1%) improvement with autoscaling enabled under dynamic workloads.

1. Introduction

Large Language Models (LLMs) have revolutionized artificial intelligence, creating an urgent demand for scalable inference serving systems. However, the operational cost of serving LLMs with billions of parameters (e.g., the DeepSeek-v3 (Guo et al., 2025) and Llama-4 (Grattafiori

¹The Hong Kong University of Science and Technology, Hong Kong SAR, China ²Tsinghua University, Beijing, China ³Shanghai Jiao Tong University, Shanghai, China. Correspondence to: Binhang Yuan <biyuan@ust.hk>.

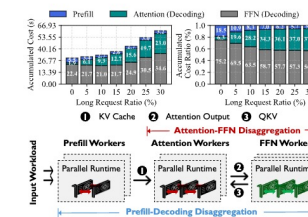


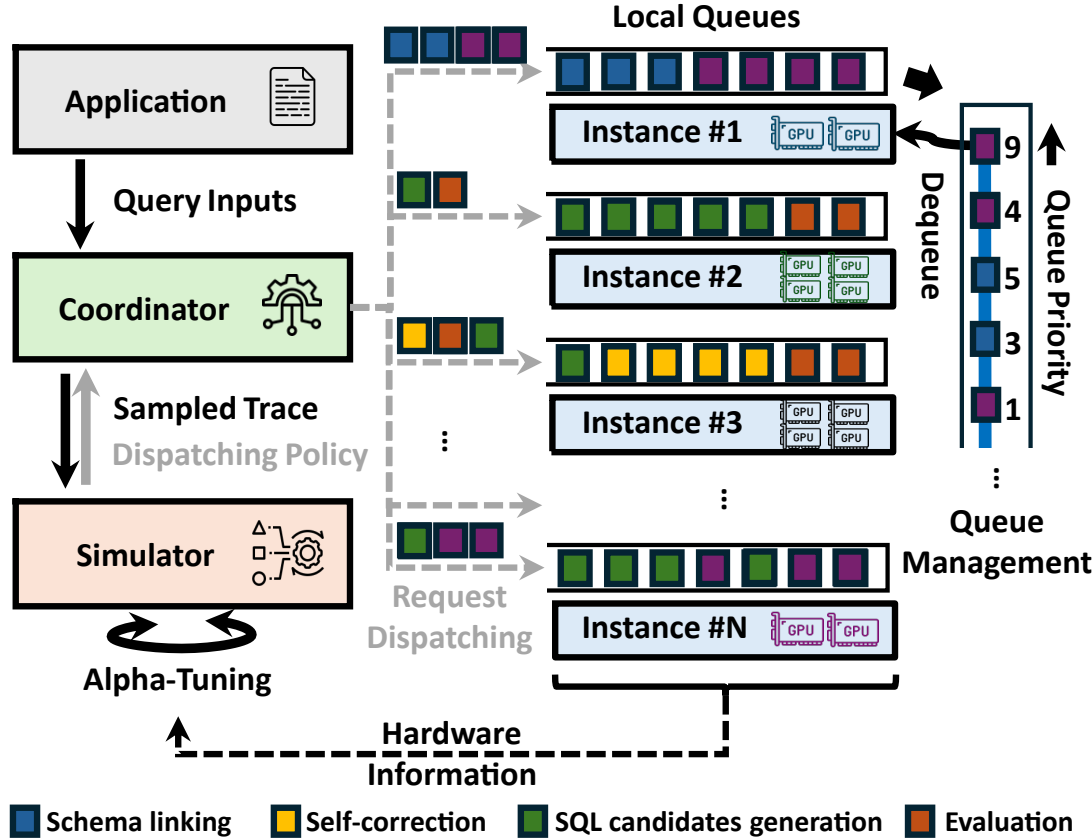
Figure 1. Upper: Attention and FFN loads change according to different long (≈10k tokens (An et al., 2024)) versus short (≈1k tokens (Patel et al., 2024)) request mix ratios. When incorporating 30% long requests, the prefill cost decreases 1.7× (a 3.5× decrease in ratio); the attention cost surges 12× (a 6× increase in ratio); while the FFN cost rises only 1.5× (a 1.3× decrease in ratio). Lower: Exampled architecture of a fully disaggregated system.

et al., 2024)) remains prohibitively high. A critical factor driving these inefficiencies is the *workload heterogeneity* inherent in real-world production traffic, such as the diverse mix of long versus short requests (Agrawal et al., 2024; Jiang et al., 2025a; Zhao et al., 2024b) and the varying balance of compute versus memory needs (Patel et al., 2024; Zhong et al., 2024). In this paper, we address this challenge by leveraging both a *fully disaggregated inference architecture* and *fine-grained heterogeneous resource autoscaling*.

Disaggregated inference has emerged as a promising architectural paradigm for LLM serving. LLM inference consists of two distinct phases: The *prefill* phase, which processes the entire input prompt in parallel and is compute-bound, and the *decoding* phase, which generates tokens autoregressively and is memory-bandwidth-bound due to repeated model weight loading. By separating these phases onto dedicated GPU pools, prefill-decoding disaggregation eliminates interference between their conflicting resource demands, allowing prefill workers to maximize GPU compute utilization while decoding workers can be optimized for memory throughput. Furthermore, within the decoding phase, the *attention* operation (memory-bound, scaling with sequence length) exhibits different characteristics from the

HexGen-Flow

Optimizing LLM Inference Request Scheduling for Agentic Text-to-SQL



HEXGEN-FLOW: Optimizing LLM Inference Request Scheduling for Agentic Text-to-SQL

You Peng*
HKUST
Hong Kong, China
ypengbt@connect.ust.hk

Yuehe Jiang*
HKUST
Hong Kong, China
yuehejiang@gmail.com

Wenqi Jiang
ETH Zurich
Zurich, Switzerland
wenqi.jiang@inf.ethz.ch

Chen Wang[†]
Tsinghua University
Beijing, China
wang_chen@tsinghua.edu.cn

Binhuan Yuan[†]
HKUST
Hong Kong, China
biyuan@ust.hk

Abstract—Recent advancements in leveraging the agentic paradigm of large language models (LLMs) have substantially improved Text-to-SQL capabilities, empowering users without specialized database knowledge to intuitively query databases. However, deploying agentic LLM-based Text-to-SQL systems in production presents significant challenges, stemming from their inherently multi-stage computational dependencies, strict latency requirements, and the complexity of deployment across heterogeneous GPUs widely existing in enterprise clusters. Meanwhile, existing LLM serving frameworks are primarily designed for independent inference tasks, resulting in suboptimal performance and frequent service-level objective (SLO) violations in Text-to-SQL workloads. In this paper, we introduce HEXGEN-FLOW, a novel framework designed explicitly to schedule and execute agentic multi-stage LLM-based Text-to-SQL workflows on heterogeneous GPU clusters serving multi-tenant Text-to-SQL requests. HEXGEN-FLOW introduces a hierarchical scheduling approach that combines global workload-balanced task dispatching with an adaptive local priority queue, guided by a systematic analysis of agentic Text-to-SQL workflows. Additionally, we propose a lightweight simulation-based method for tuning critical scheduling hyperparameters, further enhancing robustness and adaptability. Our evaluation on realistic Text-to-SQL benchmarks demonstrates that HEXGEN-FLOW significantly outperforms state-of-the-art LLM serving frameworks. Across all traces, HEXGEN-FLOW reduces P95 tail latency by 1.42~1.56× and increases throughput by 1.49~1.81×, demonstrating robust improvements under diverse workloads.

Index Terms—Text-to-SQL, LLM, Inference Scheduling.

I. INTRODUCTION

Recent advances in large language models (LLMs) have substantially improved Text-to-SQL, i.e., translating natural language questions into executable SQL statements [1]–[4]. This progress can democratize database access by enabling non-experts to query structured data without writing SQL. In enterprise settings, agentic Text-to-SQL can improve analytics productivity and accessibility.

A typical agentic Text-to-SQL workflow (Figure 1) consists of multiple interdependent stages: (1) *schema linking* to map query terms to the database schema; (2) *SQL candidate generation* to propose multiple SQL statements; (3) *self-correction* to execute and iteratively refine erroneous candidates; and (4)

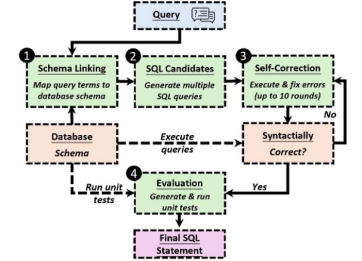


Fig. 1. A visual illustration of the agentic multiple-stage Text-to-SQL workflow with inter-stage dependency.

evaluation to select a final statement using LLM generated unit tests. Compared to non-agentic Text-to-SQL methods with a small, fixed number of LLM calls, this agentic pipeline improves quality via verifier-guided refinement and candidate selection, but introduces a longer, dependency-constrained sequence of LLM calls. Unlike many general agentic workflows, it typically runs without human-in-the-loop, and SQL execution is on the order of milliseconds—negligible relative to end-to-end latency dominated by multi-stage LLM inference.

Deploying agentic LLM-based Text-to-SQL systems in production requires not only strong models [5]–[9] and inference-time scaling algorithms [10], [11], but also an efficient and scalable serving infrastructure. In practice, enterprise environments often rely on heterogeneous GPUs for LLM serving [1]. Therefore, the goal is to serve the multi-stage Text-to-SQL pipeline efficiently on heterogeneous clusters, achieving low end-to-end latency and high throughput.

However, efficient serving is challenging due to the work-

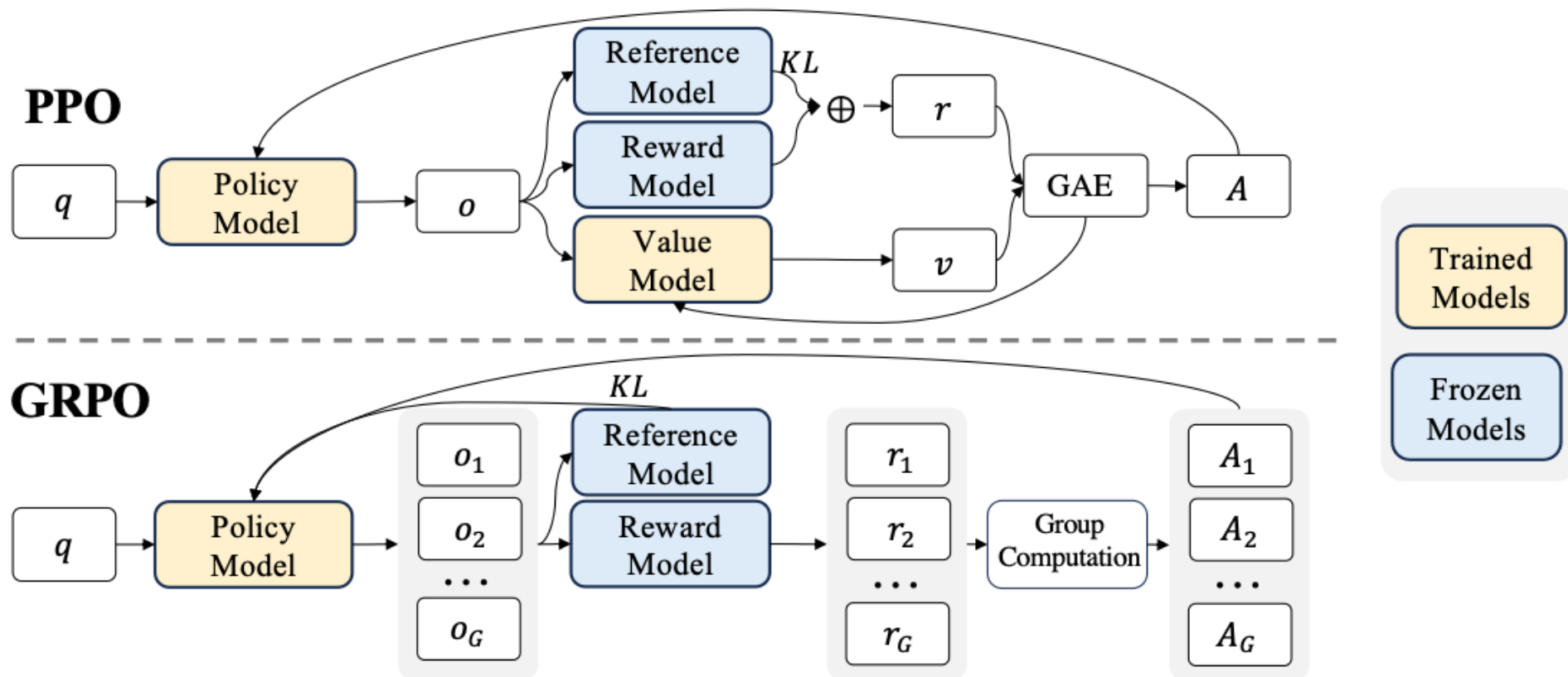
^{*}Equal contribution is indicated by *. Corresponding authors are indicated by †. This work is supported by the National Key Research and Development Program of China (2023YFB3308003), RGC-NSFC project (CRS, HKUST601/24), and WeBank-HKUST joint lab project (WEB24EG01-Q-CSE).

[†]In the current exciting era of generative AI, chip vendors typically release new generations of AI chips every 24 months. For example, Nvidia introduced the Turing architecture in 2018 [12], Ampere in 2020 [13], Hopper in 2022 [14], and Blackwell in Q4, 2024 [15]. On the other hand, one particular version of an AI chip often serves for a much longer period.



RL training for LLM over heterogeneity.

RL for LLM Post Training

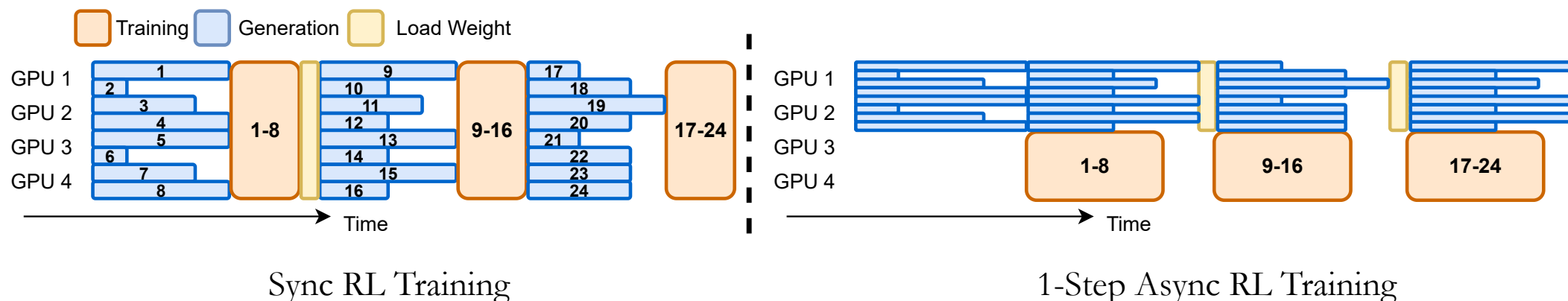


AReaL 1.0:

Asynchronous RL Is Efficient for LLM Post-Training.

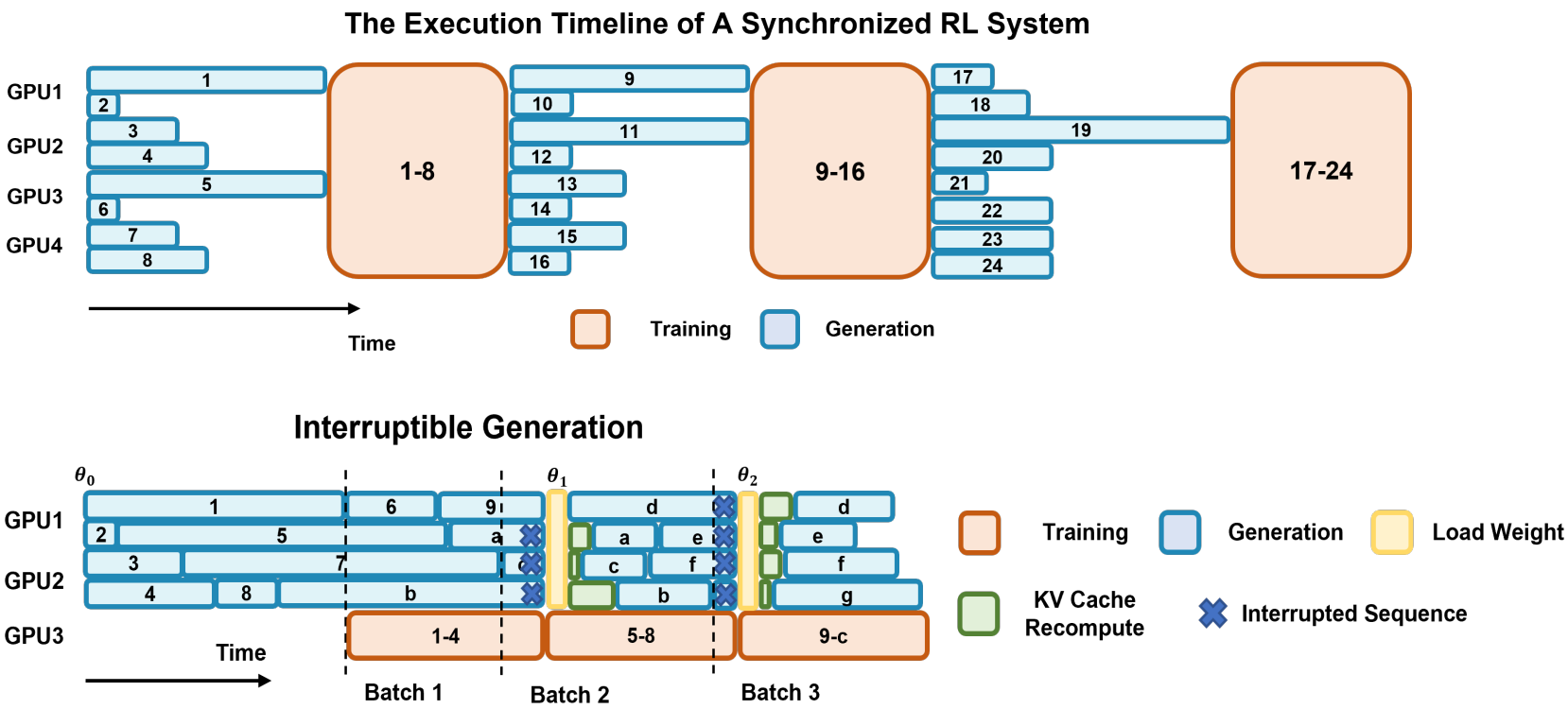
RL Training: Sync vs. Async

- **Rollout generation**: The policy LLM produces responses from a batch of prompts using generative inference.
- **Reward computation**: Using prompts and generated responses, the critic computes their values, the reference policy computes their reference log probabilities, and the reward model computes their rewards, potentially with functions executed in a sandbox.
- **Training**: The policy LLM are updated via gradient based optimization, using the batch of data produced by previous stages and the reward function.



ARealL:

Asynchronous Reinforcement Learning for Efficient and Scalable Language Reasoning



AREAL: A Large-Scale Asynchronous Reinforcement Learning System for Language Reasoning

Wei Fu^{1,2}, Jiaxuan Gao¹, Xujie Shen², Chen Zhu², Zhiyu Mei^{1,2},
Chuyi He², Shusheng Xu^{1,2}, Guo Wei², Jun Mei², Jiahu Wang³,
Tongkai Yang², Binhang Yuan³, Yi Wu¹

¹ IIIS, Tsinghua University, ² Ant Research, ³ HKUST
fuwth17@gmail.com, jxwuyi@gmail.com

Abstract

Reinforcement learning (RL) has become a trending paradigm for training large language models (LLMs), particularly for reasoning tasks. Effective RL for LLMs requires massive parallelization and poses an urgent need for efficient training systems. Most existing large-scale RL systems for LLMs are synchronous by alternating generation training in a batch setting, where the rollouts in each training batch are generated by the same (or latest) model. This stabilizes RL training but suffers from severe system-level inefficiency. Generation must wait until the longest output in the batch is completed before model update, resulting in GPU underutilization. We present AREAL, a *fully asynchronous* RL system that completely decouples generation from training. Rollout workers in AREAL continuously generate new outputs without waiting, while training workers update the model whenever a batch of data is collected. AREAL also incorporates a collection of system-level optimizations, leading to substantially higher GPU utilization. To stabilize RL training, AREAL balances the workload of rollout and training workers to control data staleness, and adopts a staleness-enhanced PPO variant to better handle outdated training samples. Extensive experiments on math and code reasoning benchmarks show that AREAL achieves **up to 2.77× training speedup** compared to synchronous systems with the same number of GPUs and matched or even improved final performance. The code of AREAL is available at <https://github.com/inclusionAI/AREAL/>.

1 Introduction

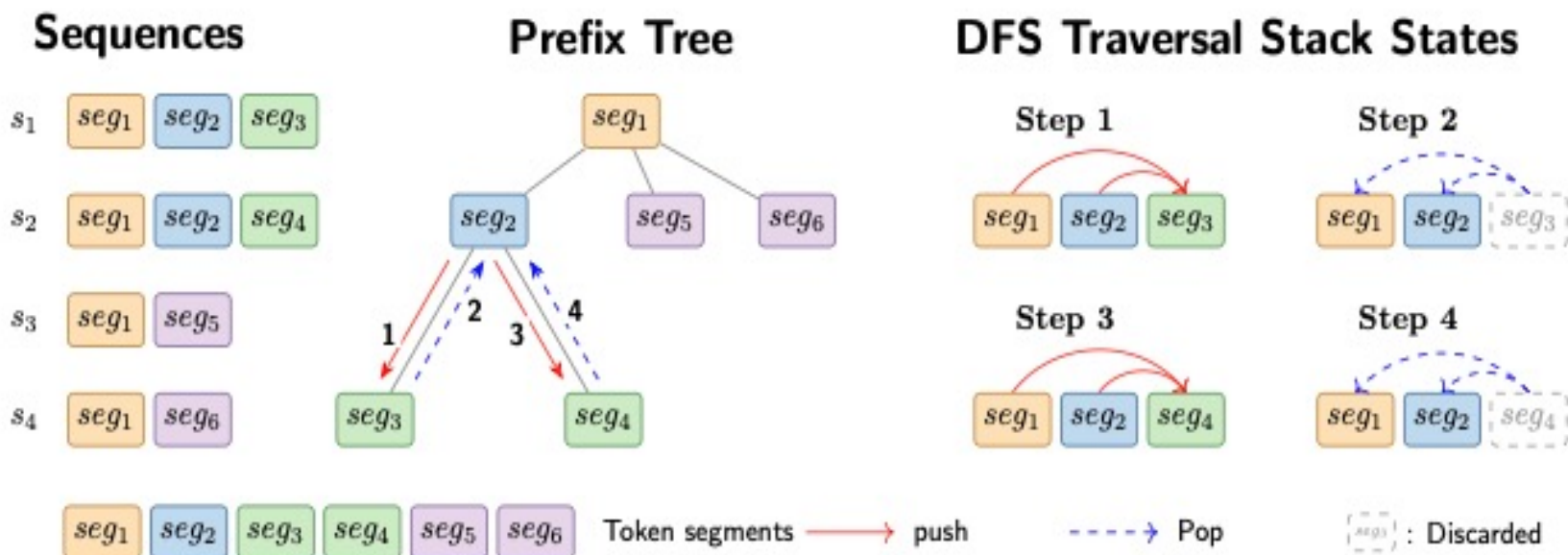
Reinforcement learning (RL) has emerged as a new scaling paradigm for enhancing the capabilities of large language models (LLMs) by enabling thinking abilities [52]. Given a prompt, RL allows an LLM to generate thinking tokens before outputting a final answer, enabling *test-time scaling* [29, 47]. These thinking LLMs are named Large Reasoning Models (LRMs) and have been shown to have particularly strong capabilities on challenging reasoning problems, such as math [9, 5, 20], coding [3, 14, 15], logic puzzles [22, 34], and agentic tasks [23, 57].

Effective RL training often requires massive parallelization to derive a large batch of rollouts for sufficient exploration, which is the key to obtaining optimal model performance. For example, popular RL algorithms, such as PPO [42] and GRPO [43], often require an effective training batch of thousands of outputs [60, 61, 53]. Moreover, an LRM can generate tens of thousands of thinking tokens for each input prompt [6], further posing an urgent need for an efficient training system to run RL training on a large scale.

[NeurIPS 2025]

AREaL-DTA:

Dynamic Tree Attention for Efficient Reinforcement Learning of Large Language Models



AREaL-DTA: Dynamic Tree Attention for Efficient Reinforcement Learning of Large Language Models

Jiarui Zhang^{1,2*}, Yuchen Yang^{2*}, Ran Yan^{1,3*}, Zhiyu Mei³, Liyuan Zhang², Daifeng Li¹, Wei Fu^{2,3}, Jiaxuan Gao^{2,3}, Shusheng Xu³, Yi Wu², Binhang Yuan¹

¹HKUST, ²Tsinghua University, ³AREaL Team, Ant Group

*Equal contribution

Abstract

Reinforcement learning (RL) based post-training for large language models (LLMs) is computationally expensive, as it generates many rollout sequences that could frequently share long token prefixes. Existing RL frameworks usually process these sequences independently, repeatedly recomputing identical prefixes during forward and backward passes during policy model training, leading to substantial inefficiencies in computation and memory usage. Although prefix sharing naturally induces a tree structure over rollouts, prior tree-attention-based solutions rely on fully materialized attention masks and scale poorly in RL settings. In this paper, we introduce AREaL-DTA to efficiently exploit prefix sharing in RL training. AREaL-DTA employs a depth-first-search (DFS)-based execution strategy that dynamically traverses the rollout prefix tree during both forward and backward computation, materializing only a single root-to-leaf path at a time. To further improve scalability, AREaL-DTA incorporates a load-balanced distributed batching mechanism that dynamically constructs and processes prefix trees across multiple GPUs. Across the popular RL post-training workload, AREaL-DTA achieves up to $8.31\times$ in τ^2 -bench higher training throughput.

1 Introduction

Post-training large language models (LLMs) with reinforcement learning (RL) is often computationally intensive [1, 2]. RL post-training workflows often require generating many similar rollout sequences for each prompt or environment state to explore different outcomes or gather sufficient reward signals [3, 4]. Crucially, these sequences frequently share long prefixes — for example, multiple candidate responses may begin with the same user prompt or initial dialogue turns [5]. In current practice, each sequence is processed independently, leading to redundant computations of the same prefix across rollouts during the training of the policy model. This repetition incurs high computational cost and memory overhead, as the training of the policy model needlessly re-computes identical token prefixes in each forward and backward pass, which results in a major bottleneck that slows training throughput and inflates GPU memory usage. In this paper, we want to explore how to effectively leverage this prefix sharing paradigm to improve RL training efficiency.

Prefix sharing is ubiquitous in modern RL training workflows for LLMs. For instance, RL for advanced reasoning LLMs and multi-turn agent training often sample multiple continuations per context, all starting with the same prompt or chain-of-thought reasoning steps [3, 4, 6, 7]. These branching trajectories naturally form a prefix tree structure: they share a common stem (prefix) before diverging into different outcomes. Yet, vanilla RL training pipelines fail to exploit this structure — they treat each branch as a separate sequence. As a result, the overlapping prefix computations are performed multiple times, wasting compute budget and memory

[ICML 2026]

AREAL-HEX:

Accommodating Asynchronous RL Training over Heterogeneous GPUs



- Computation by itself is heterogeneous:
 - LLM inference is still I/O bounded;
 - LLM Training is computation bounded.
- The communication volume is significantly reduced compared with standard pretraining:
 - Inference workers only needs to fetch weights after the training workers conducted SGD updates.

AREAL-HEX: Accommodating Asynchronous RL Training over Heterogeneous GPUs

Ran Yan^{1*}, Youhe Jiang^{1*}, Tianyuan Wu¹, Jiaxuan Gao², Zhiyu Mei³, Wei Fu², Haohui Mai¹, Wei Wang¹, Yi Wu², Binhang Yuan¹

¹HKUST, ²Tsinghua University, ³Ant Group

*Equal contribution

Abstract

Maximizing the training throughput and cost-efficiency of reinforcement learning (RL) for large language models (LLMs) is essential to democratize this advanced technique. One promising but challenging approach is to deploy such a computational workflow over heterogeneous GPU clusters. Unlike conventional large-scale LLM pretraining, RL training generally decomposes into three coupled stages, i.e., rollout generation, reward computation, and policy/value updates, which exhibit markedly different compute intensities, memory footprints, and communication patterns. Recent research shows that fully asynchronous RL training can disaggregate these stages across disjoint hardware pools without sacrificing training stability, creating a great opportunity for real-world heterogeneous deployment. Towards this end, we present AREAL-HEX, a heterogeneity-aware asynchronous RL training system that effectively schedules how to execute rollout generation and policy model training over heterogeneous GPU clusters while enforcing data staleness bounds. Concretely, AREAL-HEX uses a two-phase scheduler: (i) a constrained search with mixed-integer linear programming to select per-stage parallelization strategies and workload assignments given a resource budget, and (ii) a graph-partitioning step that allocates heterogeneous GPUs and interconnects to maximize end-to-end throughput. Built atop a fully asynchronous RL architecture, AREAL-HEX maps HBM-I/O-bound generation and compute-bound optimization to more cost-efficient resources and balances their producer-consumer interactions to avoid both idleness and stale rollout trajectories. On the mathematical reasoning task with various model scales (1.5B, 7B, and 14B), compared to homogeneous deployments of state-of-the-art asynchronous RL systems: (i) When maintaining the same total budgets, AREAL-HEX delivers up to $1.50\times$ higher training throughput; (ii) When achieving the same training throughput, AREAL-HEX results in up to $1.46\times$ reduction in training cost.

1 Introduction

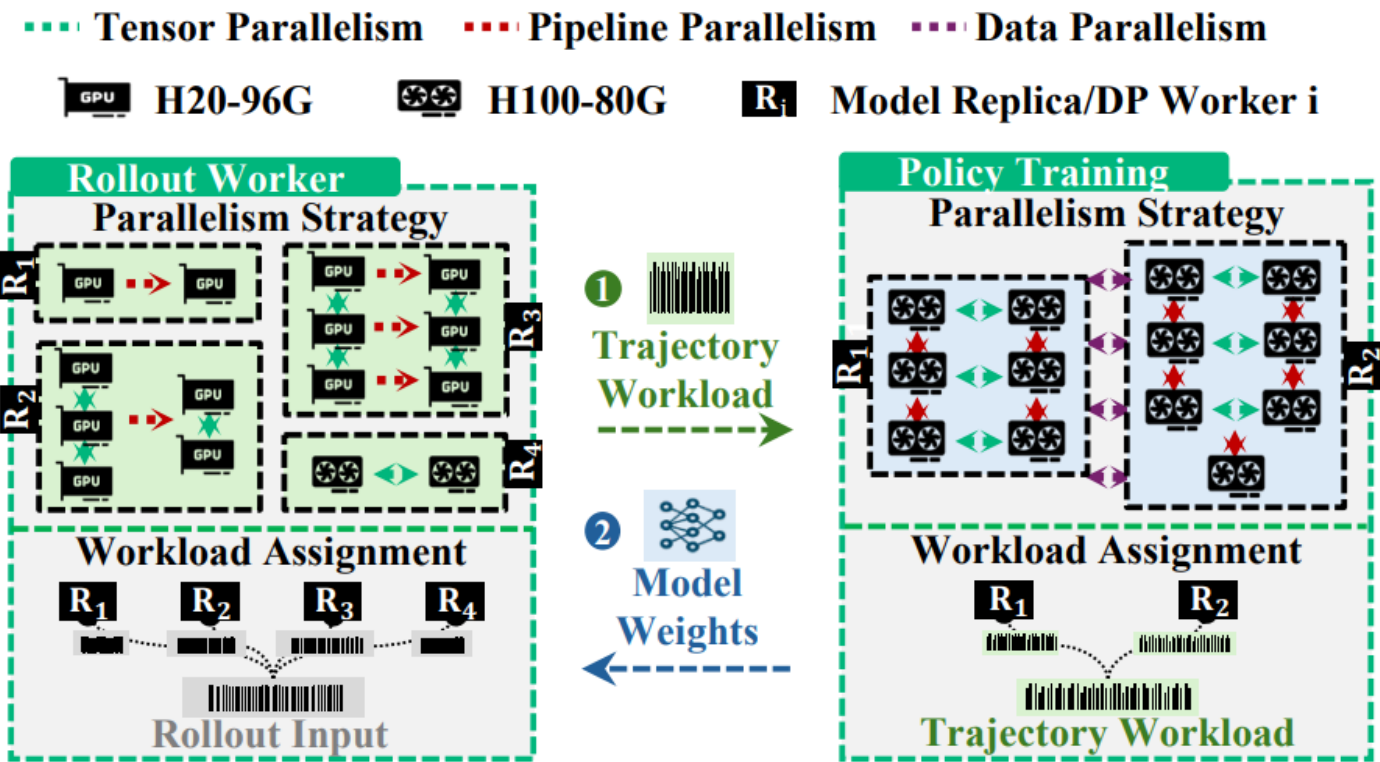
Reinforcement learning (RL) has emerged as a popular approach for enhancing large language models (LLMs) across various tasks [1], including math [2], code generation [3], agentic tool use [4], etc, which becomes essential to advance the development of AI. Unlike conventional large-scale pretraining by standard stochastic gradient-based optimization, RL training comprises multiple coupled stages, where each stage includes heterogeneous compute intensities, memory footprints, and communication patterns [5]; furthermore, recent study suggests that asynchronous RL paradigm can effectively disaggregate these different stages over different set of hardware without compromising the stability in RL training [6, 7]. Such an intrinsic computational heterogeneity under the disaggregated infrastructure naturally provides a great opportunity to deploy such a

[Arxiv. 2511.00796]

arXiv:2511.00796v1 [cs.DC] 2 Nov 2025

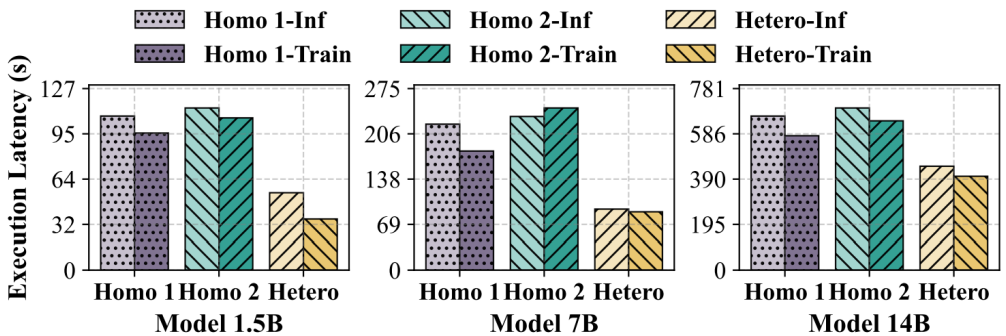
AReaL-Hex

Accommodating Asynchronous RL Training over Heterogeneous GPUs



Model	System	Throughput	Cost
1.5B model	AREAL (H800)	1.84×10^4 t/s	\$126.72/h
	AREAL-HEX	1.79×10^4 t/s	\$86.64/h
7B model	AREAL (H800)	5.98×10^3 t/s	\$126.72/h
	AREAL-HEX	6.00×10^3 t/s	\$86.64/h
14B model	AREAL (H800)	2.16×10^3 t/s	\$126.72/h
	AREAL-HEX	2.30×10^3 t/s	\$86.64/h

Cost with the same end-to-end throughput

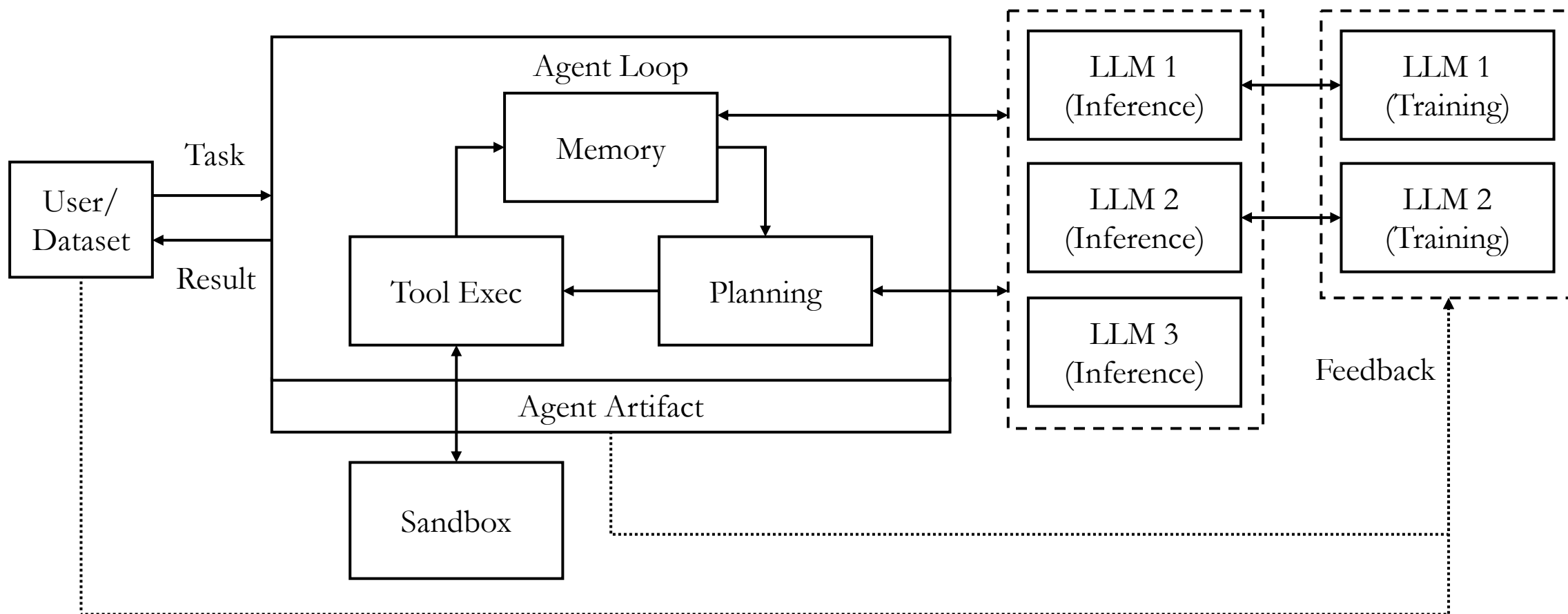


Throughput breakdown under the same cost

AReaL 2.0:

Agent-centric Platform to Support Flexible Online RL.

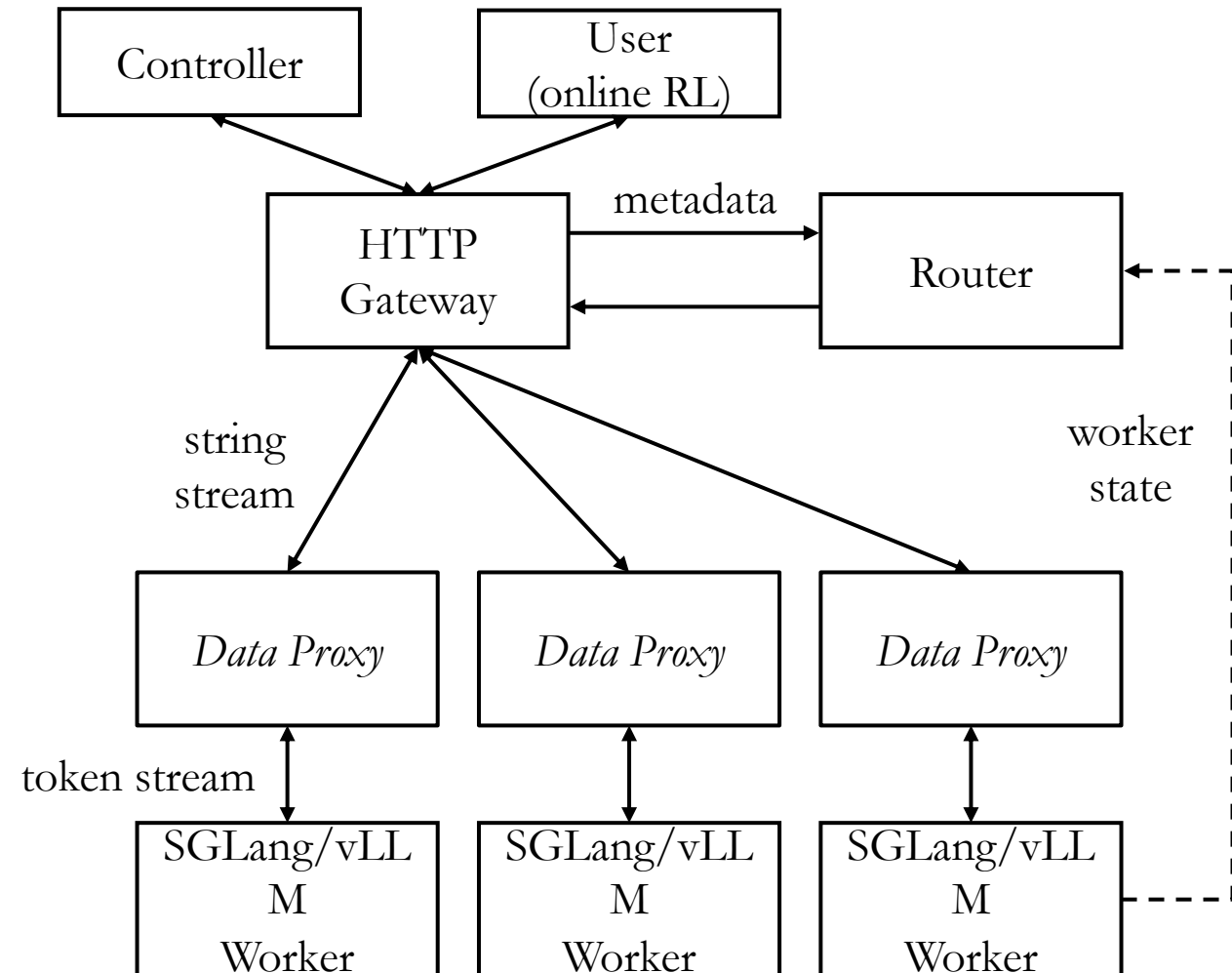
Agent-Centric Online RL Infra



AReaL 2.0 Inference:

Agentic Inference with Comprehensive Data Proxy

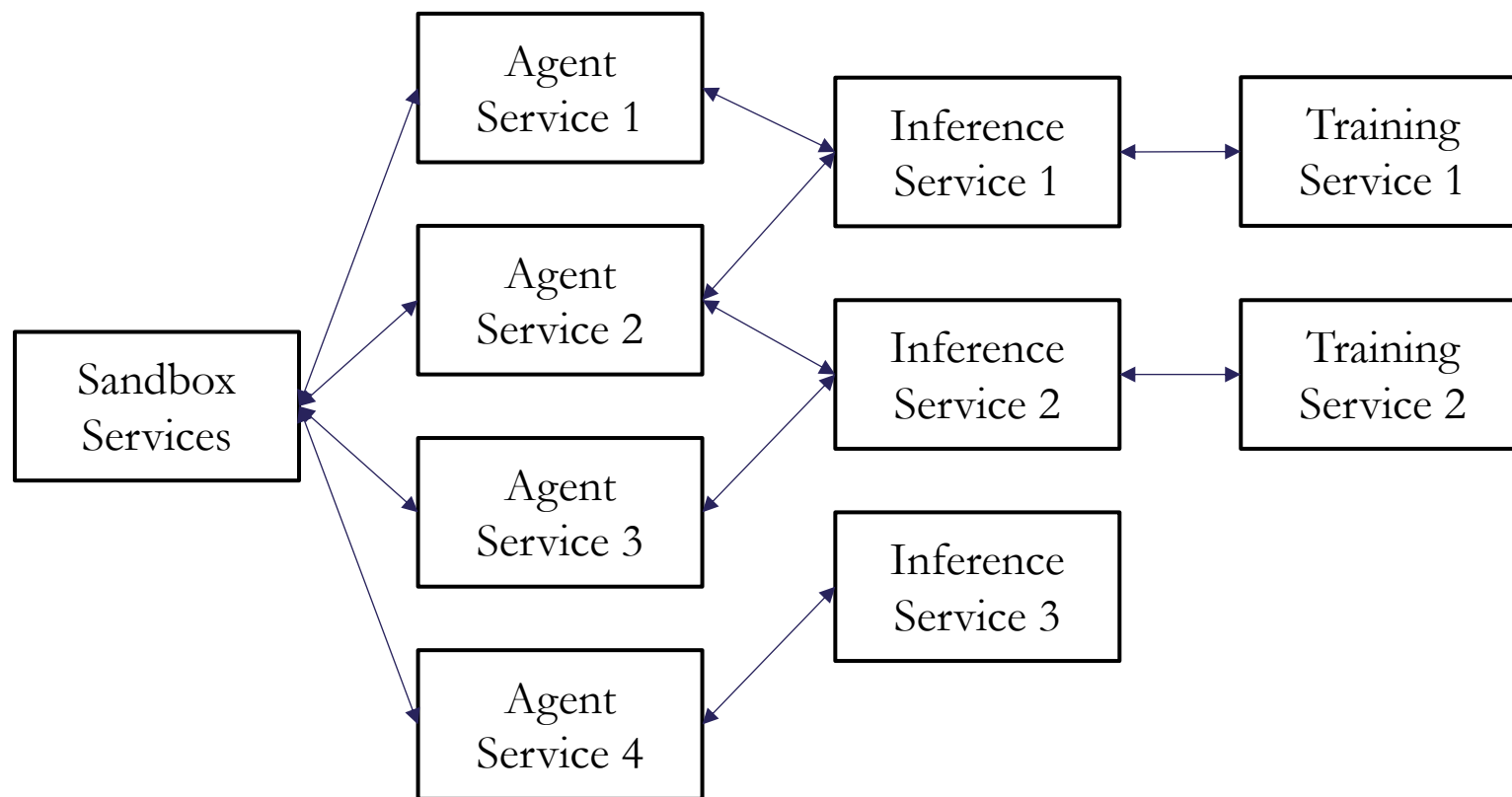
- Routing:
 - Route requests to different workers.
- Data proxy
 - Agent session lifecycle.
 - Tokenization consistency.
 - Rollout interruption.
 - Backend compatibility (vllm/sglang).
 - Data storing and transmission.



AReaL 2.0 Training:

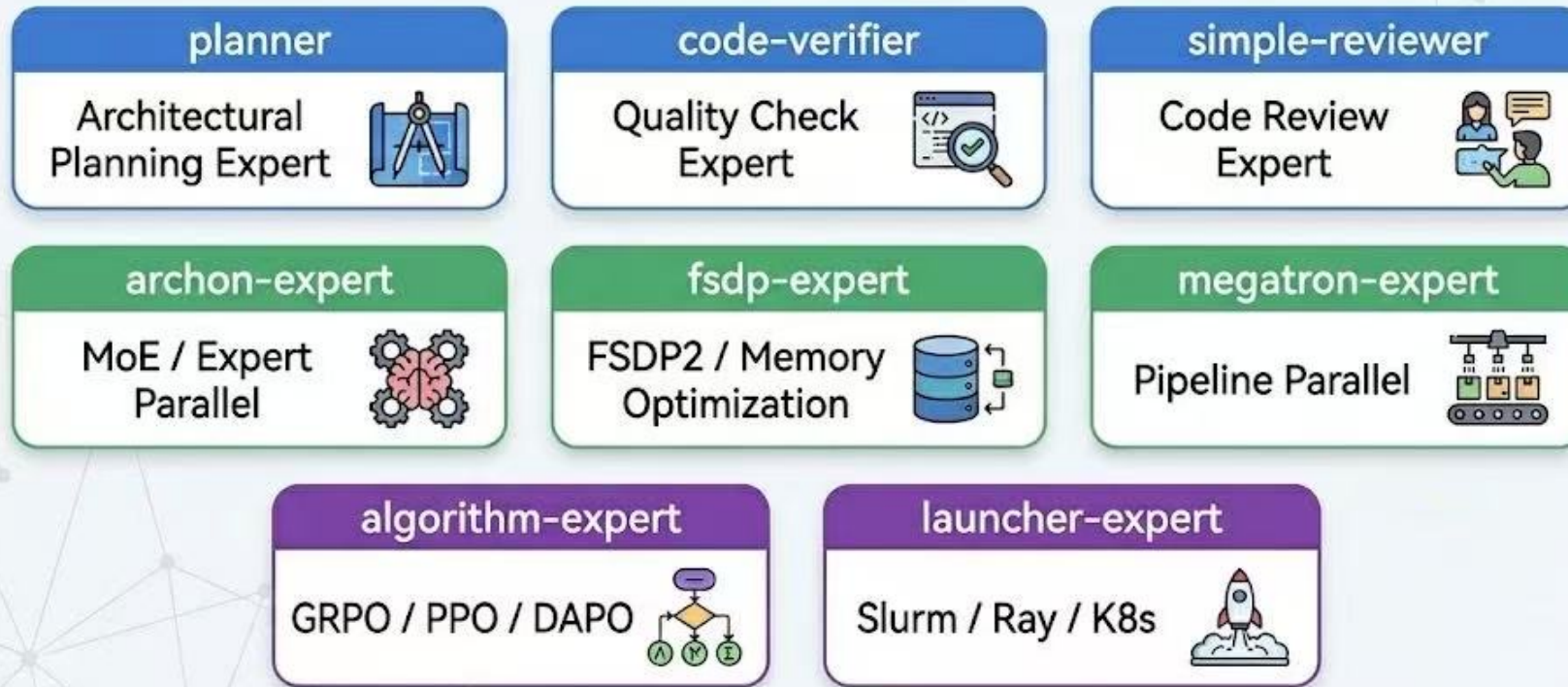
RL Training as a Light-weight Flexible Dynamic Micro-Service

- RL training:
 - A lightweight online micro-service.
- Online RL paradigms:
 - Training worker as plugin on demand.
 - Multi-agent online RL training.
 - In-context RL for agent service.
 - On policy distillation.

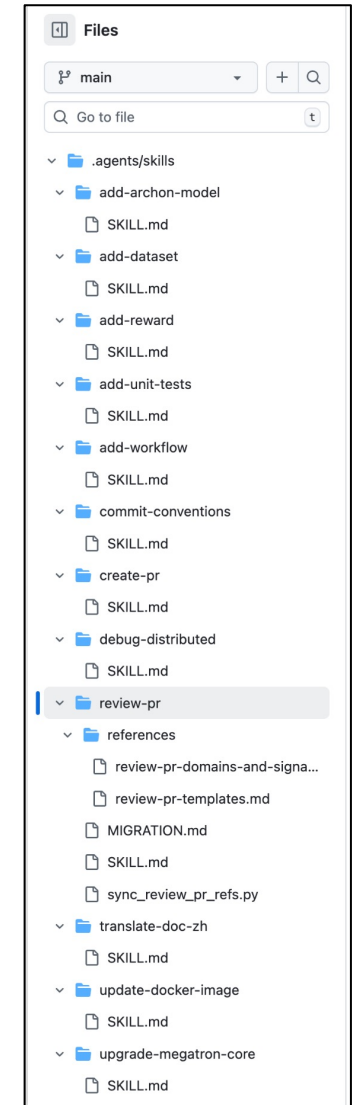


AI Assisted Development

AReal AI Expert Team

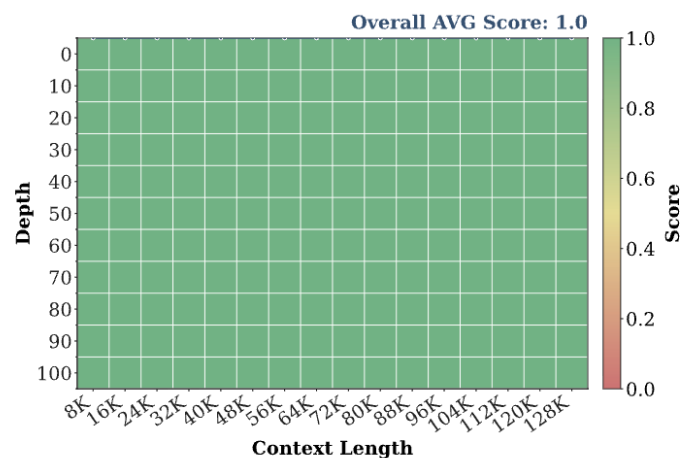


AI-Assisted Programming Paradigm in AReal

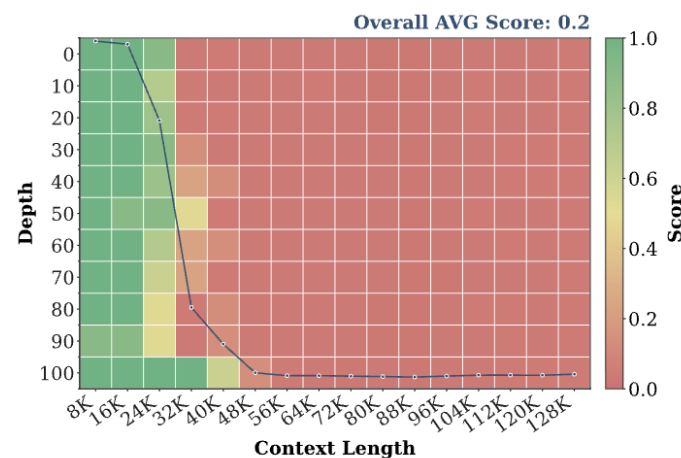


Diffusion LLM

- UltraLLaDA: Long context diffusion LLM:
 - The first to scale the context window of LLaDA to 128K.



UtraLLaDA



Training-free Baseline



UltraLLaDA: Scaling the Context Length to 128K for Diffusion Large Language Models

Guangxin He¹, Shen Nie², Fengqi Zhu², Yuankang Zhao³, Tianyi Bai¹, Ran Yan¹, Jie Fu⁴, Chongxuan Li², Binhang Yuan¹

¹HKUST, ²Renmin University of China, ³University of Chinese Academy of Sciences, ⁴Shanghai AI Lab

Abstract

Diffusion LLMs have attracted growing interest, with plenty of recent work emphasizing their great potential in various downstream tasks; yet the long-context behavior of diffusion LLMs remains largely uncharted. We present a case study of post-training techniques for extending the context window of diffusion LLMs (i.e., LLaDA) without retraining from scratch. We show that a simple modification to the standard Rotary Positional Embeddings (RoPE) extension effectively accommodates the probabilistic modeling inherent in the diffusion process, enabling stable scaling to longer context ranges. We further compare masking strategies used during post-training and analyze their impact on optimization stability and long-range recall. Instantiating these insights, we introduce ULTRALLADA, a diffusion LLM with a 128K-token context window that, in our empirical evaluation on long-context tasks, significantly outperforms training-free baselines. Our experimental results highlight the special positional extension as a key lever for scaling diffusion LLMs to extended contexts and offer practical guidance for practitioners seeking 128K-scale context via efficient post-training.

Github Repo at <https://github.com/Relaxed-System-Lab/UltraLLaDA>.

1 Introduction

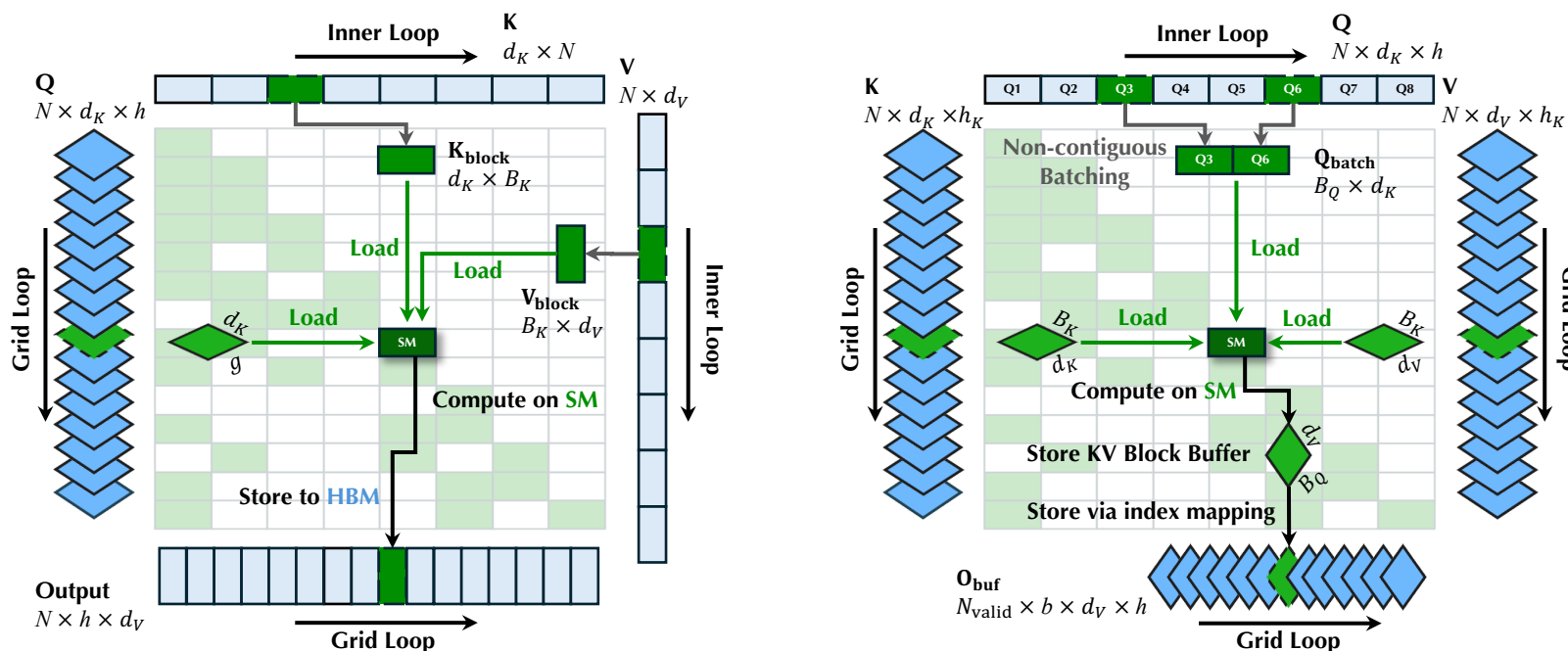
Diffusion-based large language models (LLMs) [1–3] have recently emerged as a promising new paradigm in natural language processing. Unlike popular auto-regressive LLMs [4–6], which generate text token-by-token, diffusion LLMs employ an iterative denoising process over the entire sequence, offering potential advantages over auto-regressive language models (such as bidirectional global context awareness, flexibility in conditional control, and unified modeling across modalities [7]). Substantial research has already explored the scalability [3], multimodal extensions [8, 9], reasoning [10, 11], and efficiency optimizations [12–14] of diffusion LLMs. However, one critical aspect remains largely unexplored: *how can we effectively scale and extend the context window of diffusion LLMs beyond their original training limit?* Concretely, the problem is about how to enable diffusion LLMs to handle vastly longer input sequences (e.g., up to 128K tokens) to achieve solid performance on long-context language modeling tasks.

Unlocking long-context capabilities would significantly broaden the applicability of diffusion LLMs. Many real-world language tasks (e.g. processing long documents, multi-turn dialogues, or retrieval-based question answering [15]) demand handling inputs far exceeding the few-thousand-token contexts used in standard training. If the context window of diffusion LLMs could be scaled to such lengths, we could explore if the advantage of diffusion LLM could be further amplified in maintaining coherence and handling complex dependencies across much larger texts than auto-regressive LLMs. Furthermore, preliminary observations

[ICLR 2026]

arXiv:2510.10481v1 [cs.CL] 12 Oct 2025

Flash Sparse Attention (FSA)



- Left: NSA kernel iterates query tokens in outer loop, and processes KV blocks in the inner loop: This strategy reaches kernel efficiency only when each Grouped Query Attention (GQA) group has sufficient number of query heads.
- Right: FSA kernel iterates KV blocks in the outer loop, and processes query tokens in the inner loop — partial attention results are stored in output buffer for accumulation.

Abstract

Recent advance in sparse attention mechanisms has demonstrated strong potential for reducing the computational cost of long-context training and inference in large language models (LLMs). Native Sparse Attention (NSA), one state-of-the-art approach, introduces natively trainable, hardware-aligned sparse attention that delivers substantial system-level performance boost while maintaining accuracy comparable to full attention. However, the kernel implementation of NSA forces a loop order that is only efficient with a relatively large number of query heads in each Grouped Query Attention (GQA) group, whereas existing LLMs widely adopt much smaller number of query heads in each GQA group — such an inconsistency significantly limits the applicability of this sparse algorithmic advance. In this work, we propose Flash Sparse Attention (FSA), an alternative kernel implementation that enables efficient NSA computation across a wide range of popular LLMs with varied smaller number of heads in each GQA group on modern GPUs. Compared to vanilla NSA kernel implementation, our empirical evaluation demonstrates that FSA achieves (i) up to 3.5 $\times$ and on average 1.6 $\times$ kernel-level latency reduction, (ii) up to 1.25 $\times$ and 1.09 $\times$ on average end-to-end training speedup on state-of-the-art LLMs, and (iii) up to 1.36 $\times$ and 1.11 $\times$ on average for prefill-phase speedup in LLM generative inference.

GitHub Repo at <https://github.com/Relaxed-System-Lab/Flash-Sparse-Attention>.

1 Introduction

Large Language Models (LLMs) with long context windows [1–4] face prohibitive computational costs because full attention scales quadratically in both compute and memory traffic: as the sequence length N grows, each new query must access and multiply with $O(N)$ keys and values, yielding $O(N^2)$ computation and HBM IO. As sequence length increases, attention computation becomes a critical bottleneck — for instance, attention can account for 70–80% of total decoding latency at a 64k token context [5]. In extreme cases, processing a 1 million-token prompt with an 8B model can take up to 30 minutes on a single GPU [6]. These observations underscore the urgent need for more efficient attention mechanisms in long-context LLM training and inference. A promising recent direction is to exploit sparse attention, in which each token's query interacts with only a small subset of k ($k \ll N$) informative keys and values, dramatically reducing the computation load and HBM I/O volumes from $O(N^2)$ to $O(kN)$. However, implementing efficient sparse attention at scale is non-trivial — In fact, the challenge of implementing high-performance kernels has become a major obstacle to deploying state-of-the-art sparse attention techniques in practice. In this paper, we want to explore: Can we design and implement an efficient sparse attention kernel for a wide range of current LLMs to fully unleash the

[ICLR 2026]



*LLM Evaluation is essential to understand
the boundary of its potential.*

How Powerful is LLM for Atmospheric Science?

A Case Study



Prompt:

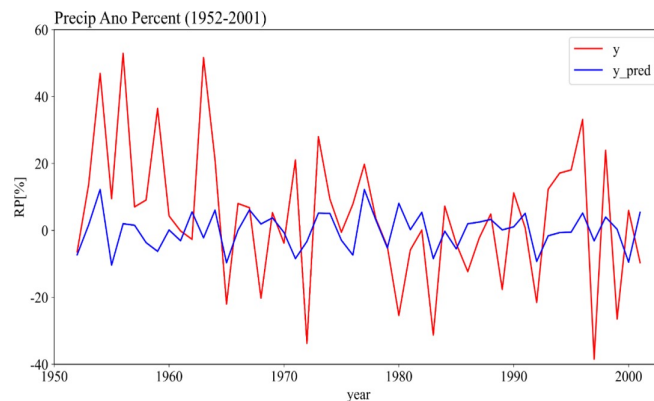
The attached file is several meteorological variables in different years. The y is Precipitation Anomaly percent in the second column, the other five columns are five x meteorological variables to predict y. Develop a statistical model using Multiple Linear Regression between y and x1 x2 x3 x4 x5, display the figure showing y and y_predict legends calculated by model.



Manual

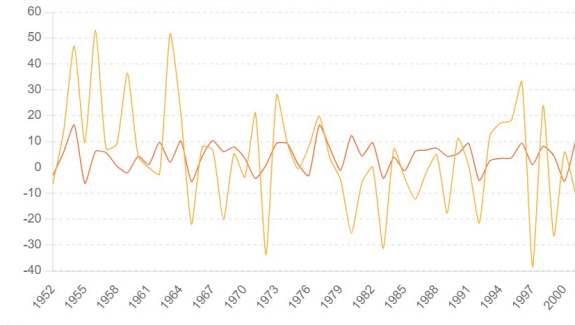


GPT-4o:



Actual Vs Predicted Precipitation Anomaly Percent

Y Precipitation Anomaly Percent by X Year for Actual Precipitation Anomaly Percent and Predicted Precipitation Anomaly Percent



Water Resources Research

COMMENTARY
10.1029/2024WR039553

Key Points:

- Foundation models are valuable tools for assisting humans in many tasks, including data processing, weather diagnosis, and decision-making.
- Concerns such as hallucinations and responsibility in the use of foundation models emphasize the need for careful implementation.
- Enhanced human-AI collaboration and the development of domain-specific foundation models are key in advancing the future of hydrometeorology.

Supporting Information:
Supporting Information may be found in the online version of this article.

Correspondence to:
M. Lu,
mengqian.lu@ust.hk

Citation:

Zhang, L., Song, Y., Cai, H., Lu, M., Li, C., Yuan, B., et al. (2025). Foundation models as assistive tools in hydrometeorology: Opportunities, challenges, and perspectives. *Water Resources Research*, 61, e2024WR039553. <https://doi.org/10.1029/2024WR039553>

Received 26 NOV 2024
Accepted 7 APR 2025

Author Contributions:

Conceptualization: Mengqian Lu, Binghang Yuan
Formal analysis: Lujia Zhang, Yurong Song, Hanzhe Cui
Funding acquisition: Mengqian Lu
Investigation: Lujia Zhang, Yurong Song, Hanzhe Cui, Mengqian Lu, Bin Wang, Yimann Lail, Jing Yang
Methodology: Lujia Zhang, Binghang Yuan
Project administration: Mengqian Lu
Software: Lujia Zhang, Yurong Song, Hanzhe Cui
Supervision: Mengqian Lu

© 2025 The Author(s).
This is an open access article under the terms of the Creative Commons Attribution-NonCommercial License, which permits use, distribution and reproduction in any medium, provided the original work is properly cited and is not used for commercial purposes.

Foundation Models as Assistive Tools in Hydrometeorology: Opportunities, Challenges, and Perspectives

Lujia Zhang^{1,2}, Yurong Song¹, Hanzhe Cui¹, Mengqian Lu¹, Chenyue Li³, Binghang Yuan¹, Bin Wang⁴, Upmannu Lall⁵, and Jing Yang⁶

¹Department of Civil and Environmental Engineering, The Hong Kong University of Science and Technology, Hong Kong, China, ²Individual Interdisciplinary Program (Earth, Ocean and Atmospheric Science), Interdisciplinary Program Office, The Hong Kong University of Science and Technology, Hong Kong, China, ³Department of Computer Science and Engineering, The Hong Kong University of Science and Technology, Hong Kong, China, ⁴Department of Atmospheric Sciences and International Pacific Research Center, University of Hawaii's at Manoa, Honolulu, HI, USA, ⁵School of Complex Adaptive Systems, & Water Institute, Arizona State University, Tempe, AZ, USA, ⁶State Key Laboratory of Earth Surface Processes and Hazards Risk Governance, Faculty of Geographical Science, Beijing Normal University, Beijing, China

Abstract Most state-of-the-art AI applications in hydrometeorology are based on classic deep learning approaches. However, such approaches cannot automatically integrate multiple functions to construct a single intelligent agent, as each function is enabled by a separate model trained on independent data sets. Foundation models (FMs), which can process diverse inputs and perform different tasks, present a substantial opportunity to overcome this challenge. In this commentary, we evaluate how three state-of-the-art FMs, specifically GPT-4o, Claude 3.5 Sonnet, and Gemini 1.5 Pro, perform across four key task types in hydrometeorology: data processing, event diagnosis, forecast and prediction, and decision-making. The models perform well in the first two task types and offer valuable information for decision-makers but still face challenges in generating reliable forecasts. Moreover, this commentary highlights the concerns regarding the use of FMs: hallucination, responsibility, over-reliance, and openness. Finally, we propose that enhancing human-AI collaboration and developing domain-specific FMs could drive the future of FM applications in hydrometeorology. We also provide specific recommendations to achieve the perspectives.

1. Introduction

The application of artificial intelligence (AI) techniques is driving exciting and promising advancements in uncovering the complexities of our Earth (Sun et al., 2022). Classical machine learning methods, including artificial neural networks, support vector machines, and random forests, have demonstrated remarkable success in various classification and regression tasks in geoscience (Toms et al., 2020; White et al., 2023; Xie et al., 2024). More recently, deep learning methods have achieved substantial success in forecasting across various timescales, including precipitation nowcasting (Y. Zhang, Long, et al., 2023), short- to medium-range weather prediction (Bi et al., 2023; Lam et al., 2023), long-term climate signal forecasting (Ham et al., 2019; Zhou & Zhang, 2023). Beyond purely statistical approaches, AI techniques can be integrated with established domain-specific methods such as numerical simulations (Kochkov et al., 2024), or hydrological models (Xie et al., 2024) to develop physics-informed models. These advances illustrate the transformative role of AI in hydrometeorology, demonstrating significant contributions to data management, modeling, prediction, and process understanding while promising even greater advances in the future (M. Chen et al., 2024).

Despite their successes, these models are typically designed for specific tasks with customized inputs and outputs, raising concerns about the generalizability of AI methods to diverse real-world applications, particularly under constraints of limited data and computational resources (Nguyen et al., 2023). Moreover, training on a single data set or limited data types restricts models to learning only a narrow set of explicit or implicit properties due to potential data homogeneity (Sun et al., 2022), whereas an ideal AI system should be able to process heterogeneous inputs and integrate sophisticated reasoning procedures. Such requirements go beyond the scope of classic supervised or semi-supervised machine learning techniques—addressing these challenges calls for the application of more advanced AI techniques that can dynamically adapt to diverse data types and execute complex, multi-step analytical tasks, thereby enhancing the robustness and applicability of AI in real-world hydrometeorological scenarios (Aghajanyan et al., 2022; Lu et al., 2022).

[WRR 2025]

How Powerful is LLM for Atmospheric Science?

A Comprehensive Benchmark

Category	Model	Hydro	AtmDyn	AtmosPhy	GeoPhy	PhyOcean	Overall Acc	SymStd.
Instruction Models	Gemma-2-9B-it	24.0	13.78	15.71	11.43	20.0	15.08	4.07
	Gemma-2-27B-it	56.0	29.73	45.71	41.43	37.5	36.72	5.94
	Qwen2.5-3B-Instruct	46.0	29.19	34.28	30.0	37.5	32.09	7.71
	Qwen2.5-7B-Instruct	62.0	38.92	51.43	51.43	42.5	44.78	5.12
	Qwen2.5-32B-Instruct	58.0	47.3	64.28	62.86	55.0	53.73	6.05
	Qwen2.5-72B-Instruct-Turbo	72.0	50.0	77.86	44.29	62.5	57.61	4.73
	Llama-3.3-70B-Instruct	78.0	42.94	67.14	51.91	52.5	52.11	3.73
	Llama-3.1-405B-Instruct-Turbo	72.0	48.92	64.29	57.14	62.5	55.52	6.17
	GPT-4o-mini	48.0	42.16	58.57	40.0	40.0	45.67	4.78
	GPT-4o	68.0	51.08	74.28	60.0	55.0	58.36	5.19
Reasoning Models	Gemini-2.0-Flash-Exp	90.0	58.11	68.57	77.14	62.5	64.93	4.29
	Deepseek-V3	94.0	57.3	73.57	64.28	62.5	64.48	6.28
	QwQ-32B-Preview	88.0	60.27	87.86	74.28	60.0	69.55	4.57
	Gemini-2.0-Flash-Thinking-Exp (01-21)	100.0	78.11	85.0	91.43	80.0	82.69	3.87
	GPT-o1	100.0	82.7	92.14	92.86	87.5	87.31	3.32
Math Models	Deepseek-R1	98.0	85.68	95.0	95.71	82.5	89.4	3.48
	Deepseek-Math-7B-RL	22.0	20.54	27.86	24.29	37.5	23.58	4.44
	Deepseek-Math-7B-Instruct	36.0	28.38	33.57	30.0	40.0	30.9	4.04
	Qwen2.5-Math-1.5B-Instruct	50.0	30.0	22.86	34.29	30.0	30.45	3.24
	Qwen2.5-Math-7B-Instruct	54.0	31.35	39.28	35.71	32.5	35.22	6.14
Domain-Specific Models	Qwen2.5-Math-72B-Instruct	70.0	54.87	73.57	62.86	40.0	59.85	6.27
	ClimateGPT-7B	26.0	18.65	21.43	11.43	30.0	19.7	5.25
	ClimateGPT-70B	24.0	25.4	30.0	40.0	27.5	27.91	4.45

ATMOSSCI-BENCH: Evaluating the Recent Advances of Large Language Models for Atmospheric Science

Chenyue Li
CSE, HKUST
Clear Water Bay, Hong Kong SAR
clieh@connect.ust.hk

Wen Deng
CE, HKUST
Clear Water Bay, Hong Kong SAR
wdengan@connect.ust.hk

Mengqian Lu
CE, HKUST
Clear Water Bay, Hong Kong SAR
cemlu@ust.hk

Binhang Yuan*
CSE, HKUST
Clear Water Bay, Hong Kong SAR
biyuan@ust.hk

Abstract

The rapid advancements in large language models (LLMs), particularly in their reasoning capabilities, hold transformative potential for addressing complex challenges and boosting scientific discovery in atmospheric science. However, leveraging LLMs effectively in this domain requires a robust and comprehensive evaluation benchmark. Toward this end, we present ATMOSSCI-BENCH, a novel benchmark designed to systematically assess LLM performance across five core categories of atmospheric science problems: hydrology, atmospheric dynamics, atmospheric physics, geophysics, and physical oceanography. ATMOSSCI-BENCH features a dual-format design comprising both multiple-choice questions (MCQs) and open-ended questions (OEQs), enabling scalable automated evaluation alongside deeper analysis of conceptual understanding. We employ a template-based MCQ generation framework to create diverse, graduate-level problems with symbolic perturbation, while OEQs are used to probe open-ended reasoning. We conduct a comprehensive evaluation of representative LLMs, categorized into four groups: instruction-tuned models, advanced reasoning models, math-augmented models, and domain-specific climate models. Our analysis provides some interesting insights into the reasoning and problem-solving capabilities of LLMs in atmospheric science. We believe ATMOSSCI-BENCH can serve as a critical step toward advancing LLM applications in climate services by offering a standard and rigorous evaluation framework. The source code of ATMOSSCI-BENCH is available at [https://github.com/Relaxed-System-Lab/AtmosSci-Bench].

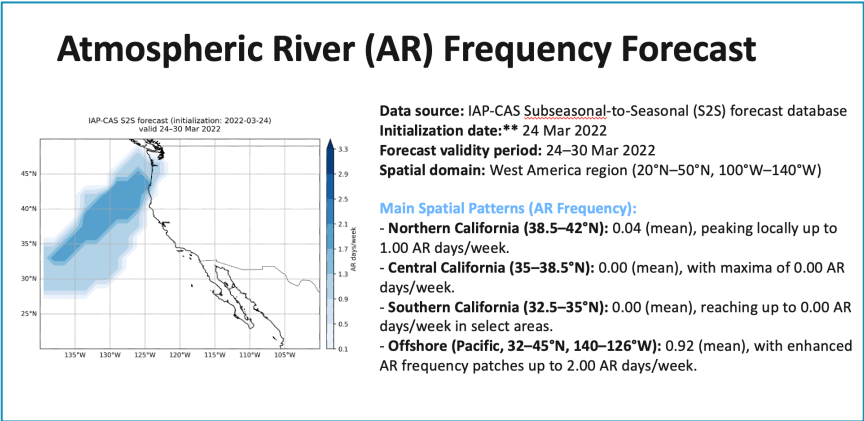
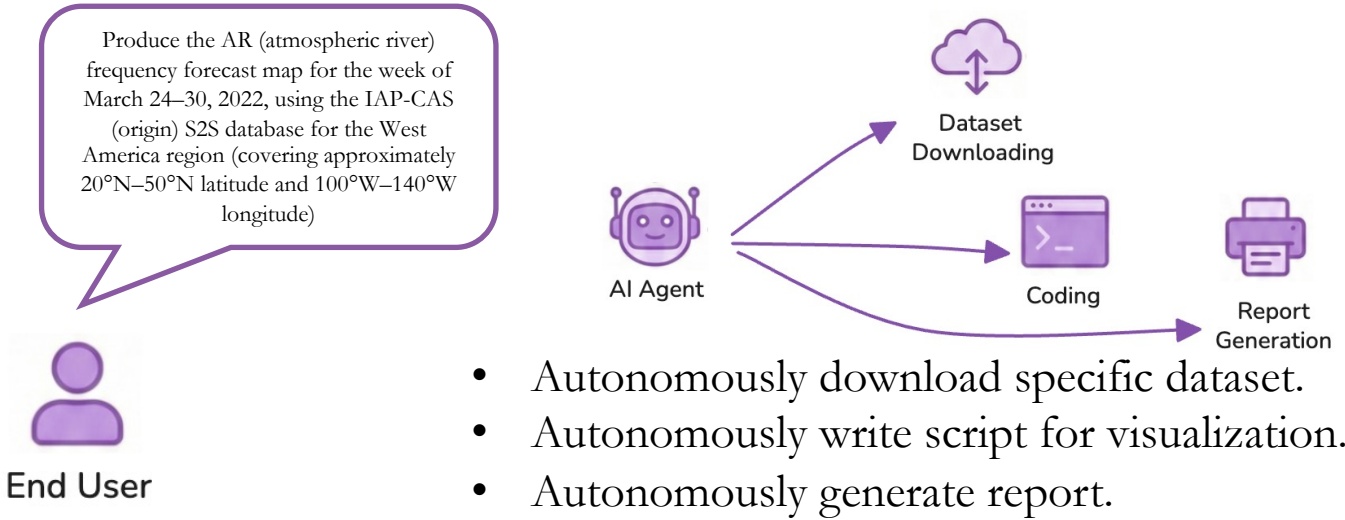
1 Introduction

Large language models (LLMs) [1], especially in their reasoning capabilities, have recently achieved remarkable progress, offering transformative potential for addressing complex challenges in atmospheric science [2, 3, 4, 5]. More recently, increasingly powerful LLMs have accelerated progress in AI4S (AI for Science), enabling a paradigm shift in scientific discovery. With their growing capabilities, LLMs show the potential to act as “AI Scientists,” partially assisting—or even autonomously conducting—hypothesis generation, experimental design, execution, analysis, and refinement [6, 7, 8, 9, 10, 11]. To advance AI for Atmospheric Science and enable the development of reliable and effective LLM-based applications for climate-related tasks, it is crucial to recognize

*Corresponding author.

How Powerful is LLM for Atmospheric Science?

LLM Climate Agent Orchestration.



CLIMATEAGENT: Multi-Agent Orchestration for Complex Climate Data Science Workflows

Hyeonjae Kim^{1*}, Chenyue Li^{1*}, Wen Deng¹, Mengxi Jin¹, Wen Huang¹, Mengqian Lu¹, Binhang Yuan^{1†}

¹The Hong Kong University of Science and Technology
*Equal contribution, [†]Corresponding author

Abstract

Climate science demands automated workflows to transform comprehensive questions into data-driven statements across massive, heterogeneous datasets. However, generic LLM agents and static scripting pipelines lack climate-specific context and flexibility, thus, perform poorly in practice. We present CLIMATEAGENT, an autonomous *multi-agent* framework that orchestrates end-to-end climate data analytic workflows. CLIMATEAGENT decomposes user questions into executable sub-tasks coordinated by an ORCHESTRATE-AGENT and a PLAN-AGENT; acquires data via specialized DATA-AGENTS that dynamically introspect APIs to synthesize robust download scripts; and completes analysis and reporting with a CODING-AGENT that generates Python code, visualizations, and a final report with a built-in self-correction loop. To enable systematic evaluation, we introduce CLIMATE-AGENT-BENCH-85, a benchmark of 85 real-world tasks spanning atmospheric rivers, drought, extreme precipitation, heat waves, sea surface temperature, and tropical cyclones. On CLIMATE-AGENT-BENCH-85, CLIMATEAGENT achieves 100% task completion and a report quality score of 8.32, outperforming GPT-4o (6.27) and a GPT-5 baseline (3.26). These results demonstrate that our multi-agent orchestration with dynamic API awareness and self-correcting execution substantially advances reliable, end-to-end automation for climate science analytic tasks.

1 Introduction

The rapid pace of climate change and the growing severity of its impacts have created an urgent need to advance data-centric climate science, which could deliver timely insights for adaptation and policy [1–3]. Automating the workflows in climate science can translate comprehensive analytical questions into executable pipelines, enabling rapid, reproducible analysis of complex environmental phenomena and supporting extreme-event forecasting, impact assessment, and adaptation planning. On the other hand, such analytic workflows need to process special climate datasets (e.g., datasets from Copernicus climate data store [4] and the European centre for medium-range weather forecasts [5]) with high volume, heterogeneity, and complexity, where intelligent automation has become essential for processing and integrating these datasets at scale [6–8]. In this paper, we aim to explore how AI agents can perform complex climate-science analytical tasks through carefully designed agentic orchestration.

Building an AI-driven climate agent is a compelling and crucial effort since the stakes of climate change demand faster and more intelligent ways to derive insights from the complex data. The accelerating pace of global change and the severity of its impacts suggest that climate scientists and policymakers urgently need timely, data-driven information for mitigation and adaptation decisions [2, 3]. At the same time, climate science workflows need to process massive and diverse datasets — from multi-model simulations to satellite

arXiv:2511.20109v1 [cs.LG] 25 Nov 2025

[TMLR 2026]

Summary

- *Accommodate LLM training over heterogeneity:*
 - To accommodate heterogeneous computation, we perform necessary system extensions and implement efficient system scheduling.
- *Accommodate LLM inference over heterogeneity:*
 - We support co-locating generative inference.
 - We also support PD-/ AFD- disaggregated generative inference.
- *Accommodate RL for LLM over heterogeneity:*
 - The asynchronous RL system is efficient.
 - We accommodate this workflow over heterogeneity and system optimizations.
 - We want to build agentic RL systems.
- *Evaluation and agent workflows for LLM application:*
 - Case studies for climate science.



Personal page:

<https://binhangyuan.github.io/site/>

Thank you!

Please Submit the Course Feedback



Thank you!

<https://sfq-survey.ust.hk/>